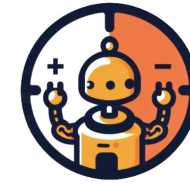




UNIVERSITÀ
DEGLI STUDI
DI PADOVA

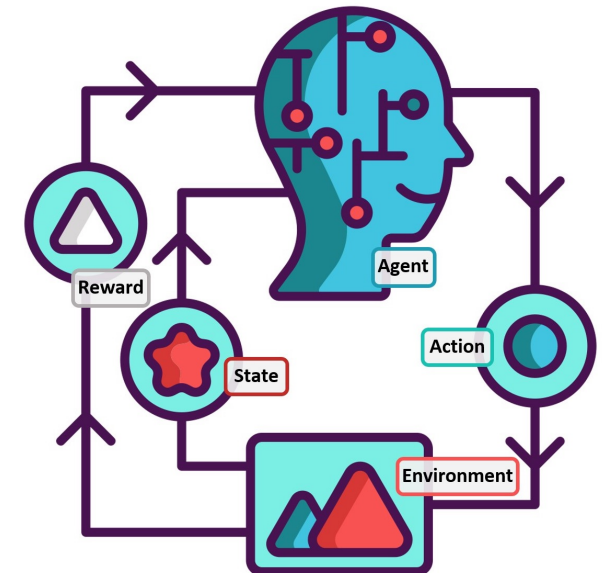
Reinforcement Learning 2024/2025



RL2
Reinforcement Learning
Research Lab @ UniPD

Lecture #09 Temporal Difference Learning

Gian Antonio Susto
Ruggero Carli



Recap: TD learning vs MC (prediction)

The main difference between MC and TD is

- MC methods wait for the actual return G_t (that is available at the end of the episode) to update the estimation of $v_\pi(s)$

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

- TD(0) uses the immediate reward (that is available after taking one action) to update the estimation of the so-called TD target

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

The TD target can be seen as the ‘best’ estimation of G_t at time $t+1$:

$$G_t = R_{t+1} + \gamma G(S_{t+1}) \sim R_{t+1} + \gamma V(S_{t+1})$$

Recap: TD learning vs MC (**prediction**)

The main difference between MC and TD is

- MC methods wait for the actual return G_t (that is available at the end of the episode) to update the estimation of $v_\pi(s)$

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

- This quantity is called the *TD error* available after taking one action, to update the estimation of the so-called **TD target**

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

The TD target can be seen as the ‘best’ estimation of G_t at time $t+1$:

$$G_t = R_{t+1} + \gamma G(S_{t+1}) \sim R_{t+1} + \gamma V(S_{t+1})$$

Recap: TD learning vs MC (**prediction**)

The main difference between MC and TD is:

- MC methods wait for the actual return (at the end of the episode) to update

Today we'll also deal with the **control** problem: any ideas on how to approach it?

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

- This quantity is called the *TD error* available after taking one action, to update the estimation of the so-called **TD target**

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

The TD target can be seen as the 'best' estimation of G_t at time $t+1$:

$$G_t = R_{t+1} + \gamma G(S_{t+1}) \sim R_{t+1} + \gamma V(S_{t+1})$$

Recap: TD learning vs MC (**prediction**)

The main difference between MC

- MC methods wait for the actual return (at the end of the episode) to update

Today we'll also deal with the **control** problem: **we'll consider Q instead of V**

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

- This quantity is called the *TD error* available after taking one action, to update the estimation of the so-called **TD target**

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

The TD target can be seen as the 'best' estimation of G_t at time $t+1$:

$$G_t = R_{t+1} + \gamma G(S_{t+1}) \sim R_{t+1} + \gamma V(S_{t+1})$$

Recap: TD learning vs MC (**prediction**)

What are the advantages/disadvantages of considering the TD error instead of the real return? what is available at the end of an episode (the return) G_t is not available until the end of the episode. The TD error is available at the end of each step.

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

- This quantity is called the *TD error* available after taking one action, to update the estimation of the so-called TD target

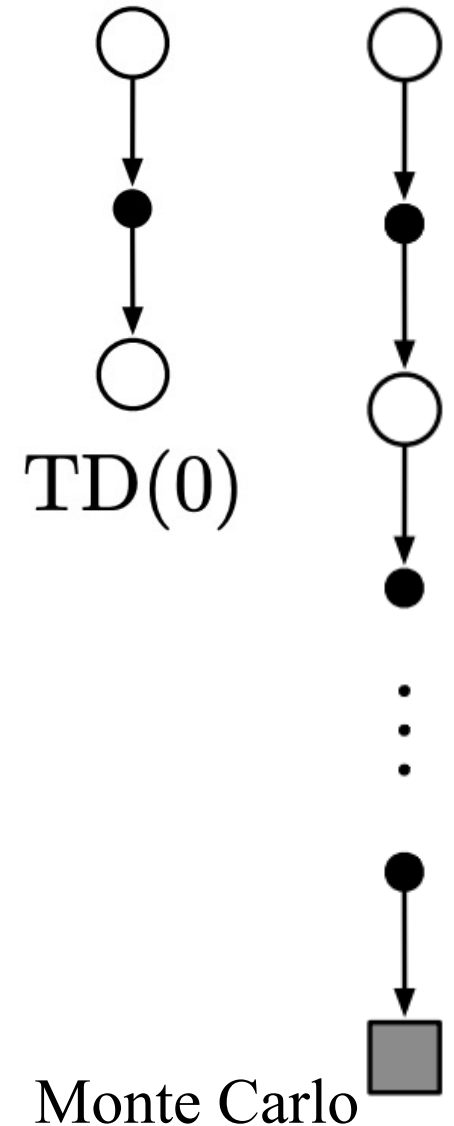
$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

The TD target can be seen as the ‘best’ estimation of G_t at time $t+1$:

$$G_t = R_{t+1} + \gamma G(S_{t+1}) \sim R_{t+1} + \gamma V(S_{t+1})$$

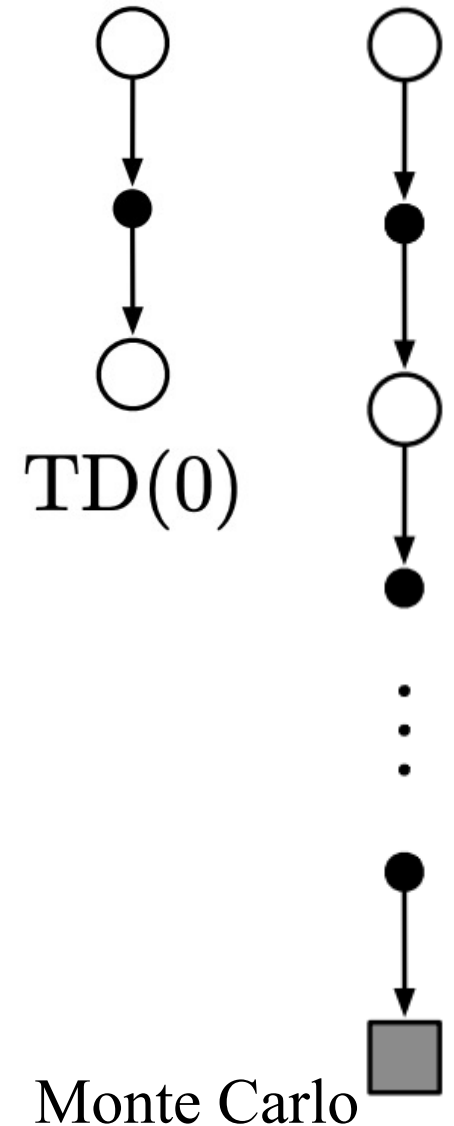
Recap: TD learning vs MC (**prediction**)

- TD methods update their estimates based on other estimates, ie. they bootstrap



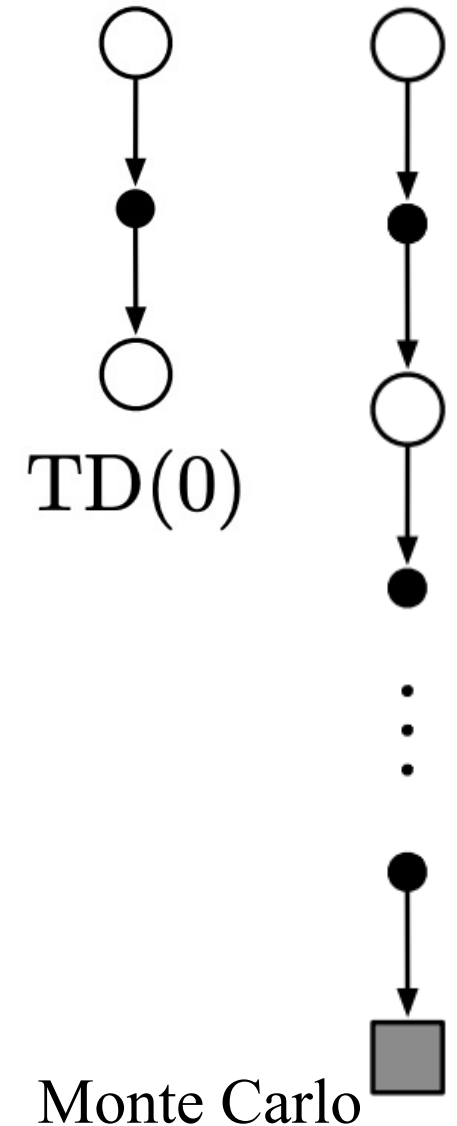
Recap: TD learning vs MC (prediction)

- TD methods update their estimates based on other estimates, ie. they bootstrap
 - TD can learn **before** knowing the final outcome:
- TD can learn online after every step / MC must wait until end of episode before return is known



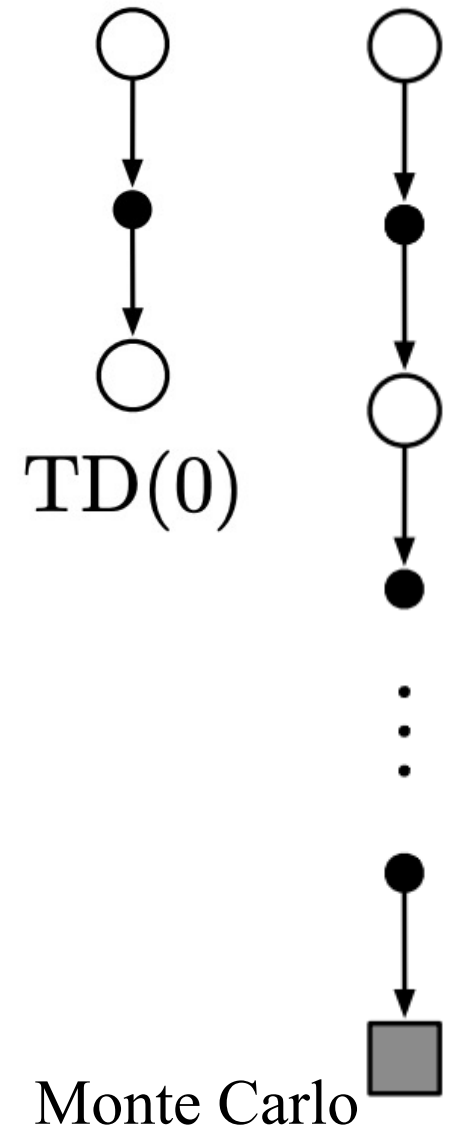
Recap: TD learning vs MC (**prediction**)

- TD methods update their estimates based on other estimates, ie. they bootstrap
- TD can learn **before** knowing the final outcome:
TD can learn online after every step / MC must wait until end of episode before return is known
- TD can learn **without** the final outcome
 - i. TD can learn from incomplete sequences / MC can only learn from complete sequences
 - ii. TD works in continuing (non-terminating) environments / MC only works for episodic (terminating) environments



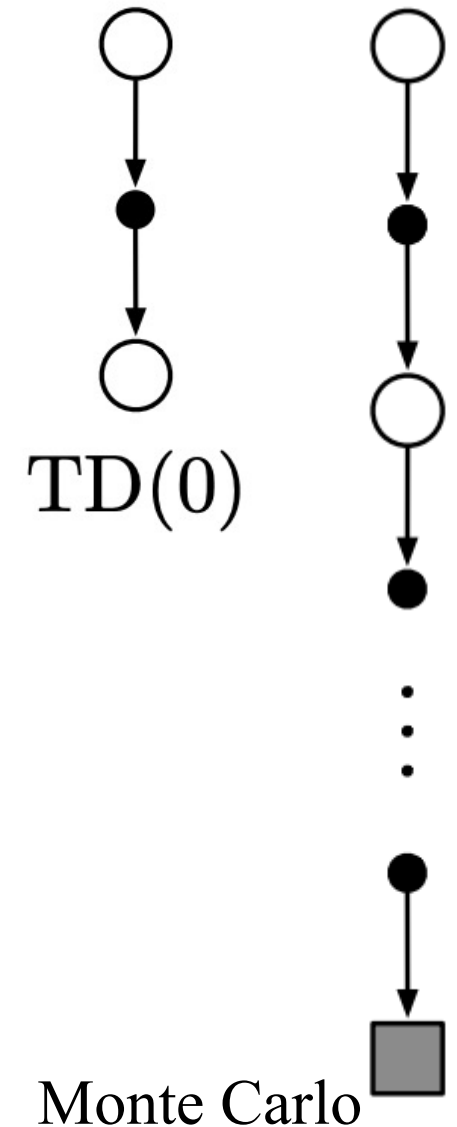
Recap: TD learning vs MC (prediction)

- TD methods update their estimates based on other estimates, ie. they bootstrap
- TD can learn **before** knowing the final outcome:
TD can learn online after every step / MC must wait until end of episode before return is known
- TD can learn **without** the final outcome
 - i. TD can learn from incomplete sequences / MC can only learn from complete sequences
 - ii. TD works in continuing (non-terminating) environments / MC only works for episodic (terminating) environments
- TD methods still converge to the right estimation of v_π



Recap: TD learning vs MC (prediction)

- TD methods update their estimates based on other estimates, ie. they bootstrap
- TD can learn **before** knowing the final outcome:
TD can learn online after every step / MC must wait until end of episode before return is known
- TD can learn **without** the final outcome
 - i. TD can learn from incomplete sequences / MC can only learn from complete sequences
 - ii. TD works in continuing (non-terminating) environments / MC only works for episodic (terminating) environments
- TD methods still converge to the right estimation of v_π
- Typically, TD approaches are **faster** to converge than MC!

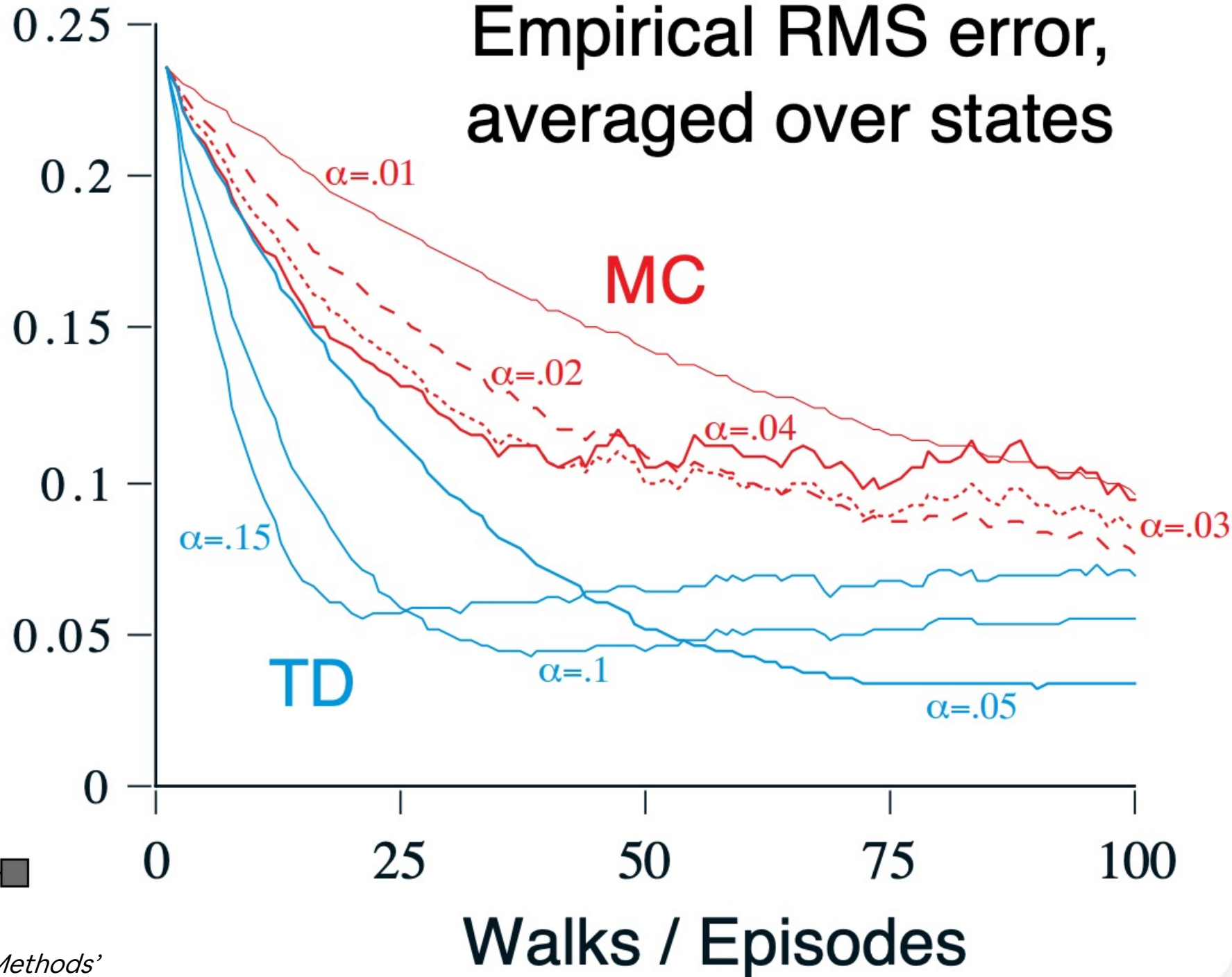
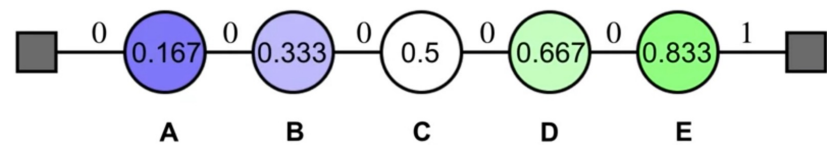


Recap

Prediction:

TD(0) – Random Walk

Estimation error -
Root Mean Square
(RMS) – averaged
over the 5 states,
the averaged over
100 runs

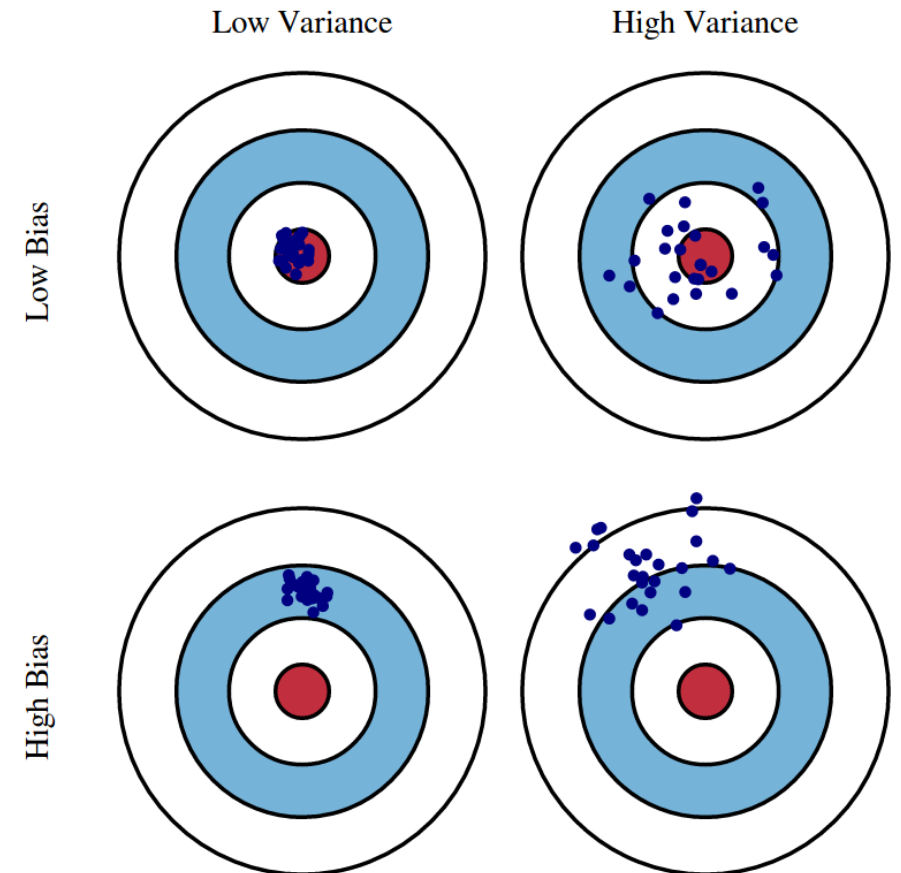
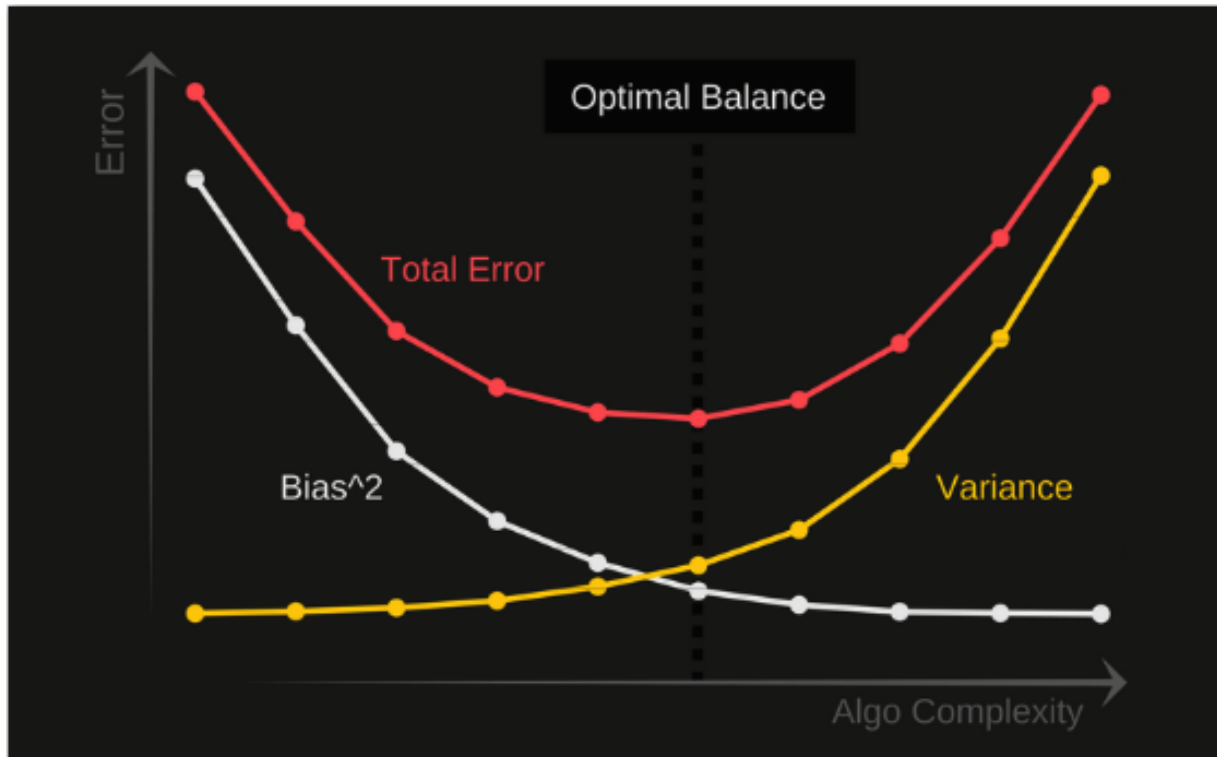


Bias/Variance Trade-off

- ?

Bias/Variance Trade-off

- A trade-off present in many Machine Learning settings



Bias/Variance Trade-off

- The return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$ is an unbiased estimate of $v_\pi(S_t)$
- True TD target $R_{t+1} + \gamma v_\pi(S_{t+1})$ (we don't know the true value function!) is an unbiased estimate of $v_\pi(S_t)$
- TD target $R_{t+1} + \gamma V(S_{t+1})$ (the target that we move to, it can be quite a wrong estimate!) is a biased estimate of $v_\pi(S_t)$

Bias/Variance Trade-off

- The return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$ is an unbiased estimate of $v_\pi(S_t)$
- True TD target $R_{t+1} + \gamma v_\pi(S_{t+1})$ (we don't know the true value function!) is an unbiased estimate of $v_\pi(S_t)$
- TD target $R_{t+1} + \gamma V(S_{t+1})$ (the target that we move to, it can be quite a wrong estimate!) is a biased estimate of $v_\pi(S_t)$
- TD target is much lower variance than the return:
 1. Return depends on **many** random actions, transitions, rewards
 2. TD target depends on **one** random action, transition, reward

Bias/Variance Trade-off

- MC has **high** variance, **zero** bias
 1. **Good convergence** properties (even with function approximation – Chapter 9, we will see this later in the course)
 2. **Not very sensitive** to initial value
 3. Very simple to understand and use
- TD has **low** variance, **some** bias
 1. Usually **more efficient** than MC
 2. TD(0) converges to $v_{\pi}(s)$ (but **not always** with function approximation -> we'll talk about this in few lectures!)
 3. More **sensitive** to initial values

(Prediction: batch MC and TD)

- If experience grows (amount of episodes increases), both MC and TD will converge to $V(s) \rightarrow v_{\pi}(s)$
- In many cases we however have limited amount of experience: a common approach is to present the experience repeatedly until the method converges upon an answer
- We repeatedly sample episode $k \in [1, K]$ and we apply MC or TD(0) to that episode
- Under batch training, MC and TD(0) converges to an ‘optimal’ solution, but with different definition of optimality

Prediction: batch MC and TD – AB Example

Two states, no discounting, 8 episodes of experience:

A, 0, B, 0

B, 1

B, 1

B, 1

B, 1

B, 1

B, 1

B, 0

What is $V(A)$ and what is $V(B)$?

Prediction: batch MC and TD – AB Example

Two states, no discounting, 8 episodes of experience:

A, 0, B, 0

B, 1

B, 1

B, 1

B, 1

B, 1

B, 1

B, 0

$$V(B) = 3/4$$

$$V(A) = ?$$

What is $V(A)$ and what is $V(B)$?

Prediction: batch MC and TD – AB Example

Two states, no discounting, 8 episodes of experience:

A, 0, B, 0

B, 1

B, 1

B, 1

B, 1

B, 1

B, 1

B, 0

$$V(B) = 3/4$$

$V(A) = 0$ is the Monte Carlo solution

-> MC converges to solution with minimum mean-squared error

What is $V(A)$ and what is $V(B)$?

Prediction: batch MC and TD – AB Example

Two states, no discounting, 8 episodes of experience:

A, 0, B, 0

B, 1

B, 1

B, 1

B, 1

B, 1

B, 1

B, 0

What is $V(A)$ and what is $V(B)$?

$$V(B) = 3/4$$

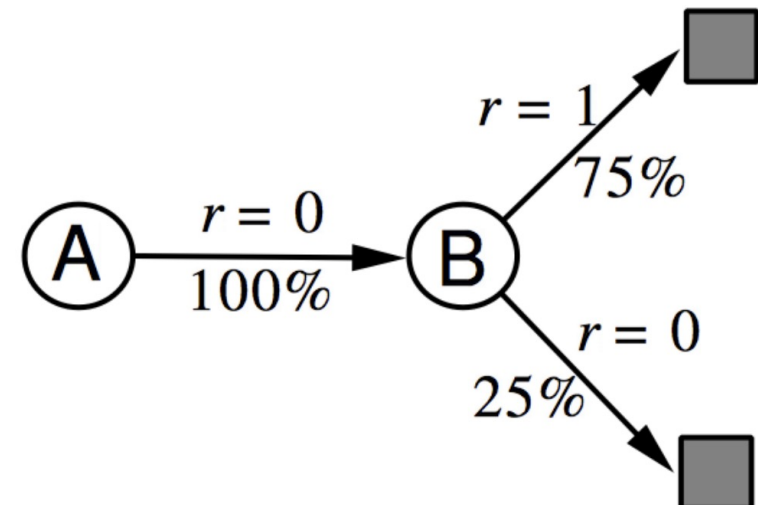
$V(A) = 0$ is the Monte Carlo solution

-> MC converges to solution with minimum mean-squared error

$V(A) = 3/4$ is the TD(0) solution

-> TD(0) (implicitly) converges to solution of max likelihood

Markov model



Prediction: MC and TD – AB Example

MC converges to solution with minimum mean-squared error: best fit to the observed returns

$$\sum_{k=1}^K \sum_{t=1}^{T_k} (G_t^k - V(s_t^k))^2$$

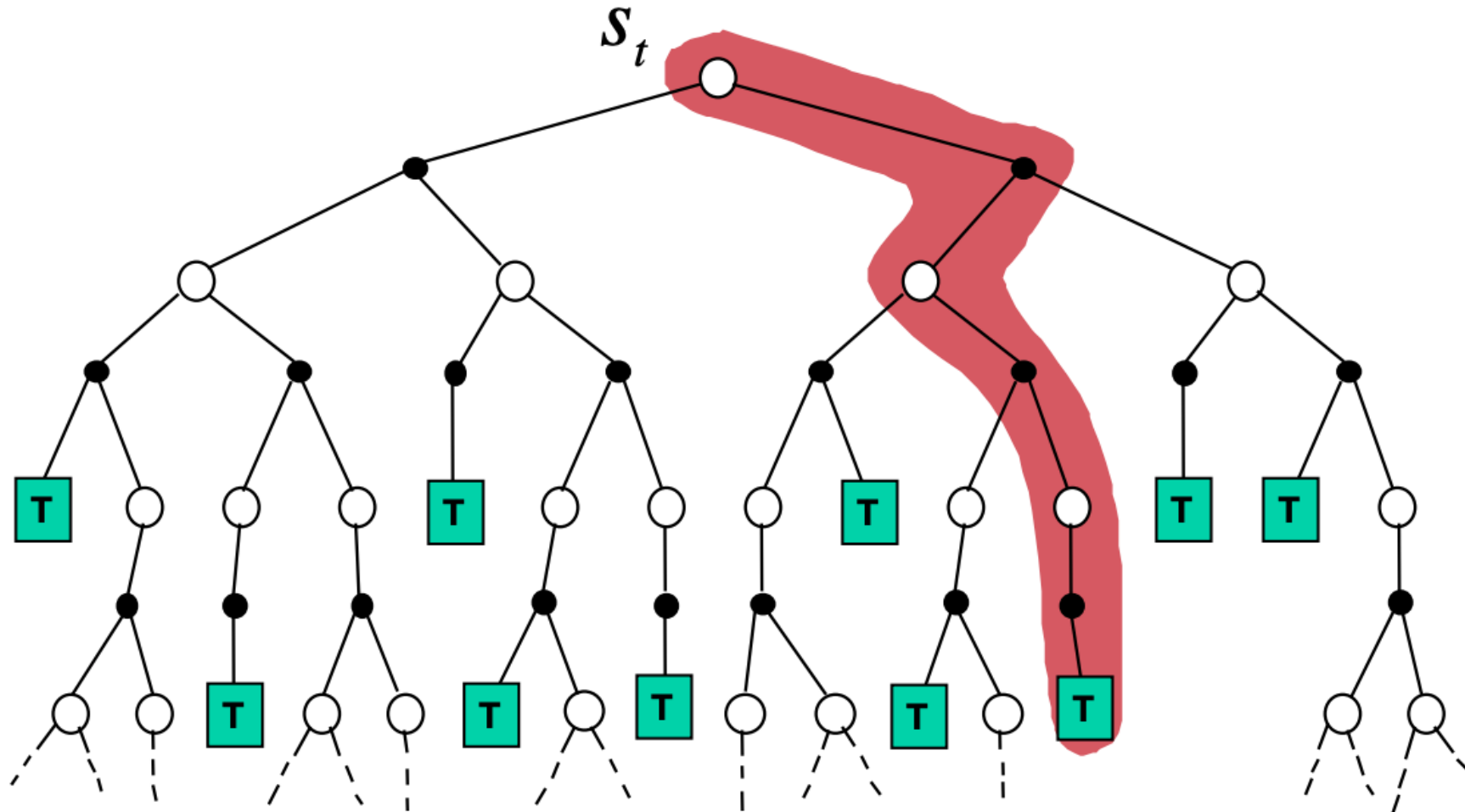
TD(0) (implicitly) converges to solution of max likelihood Markov model, ie the solution to the MDP $\langle \mathcal{S}, \mathcal{A}, \hat{\mathcal{P}}, \hat{\mathcal{R}}, \gamma \rangle$ that best fits the data

$$\hat{\mathcal{P}}_{s,s'}^a = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^{T_k} \mathbf{1}(s_t^k, a_t^k, s_{t+1}^k = s, a, s')$$
$$\hat{\mathcal{R}}_s^a = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^{T_k} \mathbf{1}(s_t^k, a_t^k = s, a) r_t^k$$

TD exploits Markov property and it is usually more efficient in Markov environment

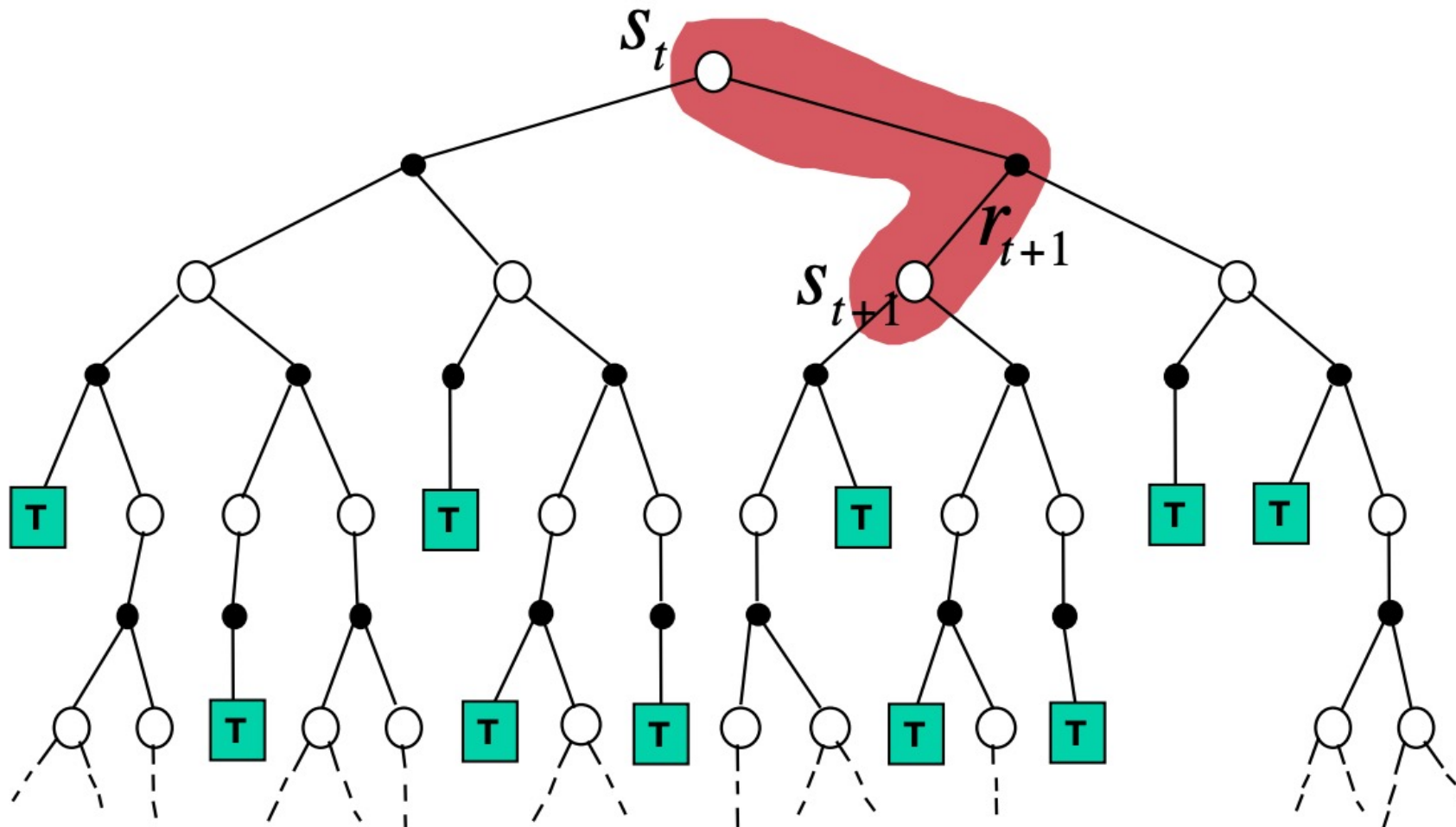
Monte-Carlo Backup

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$



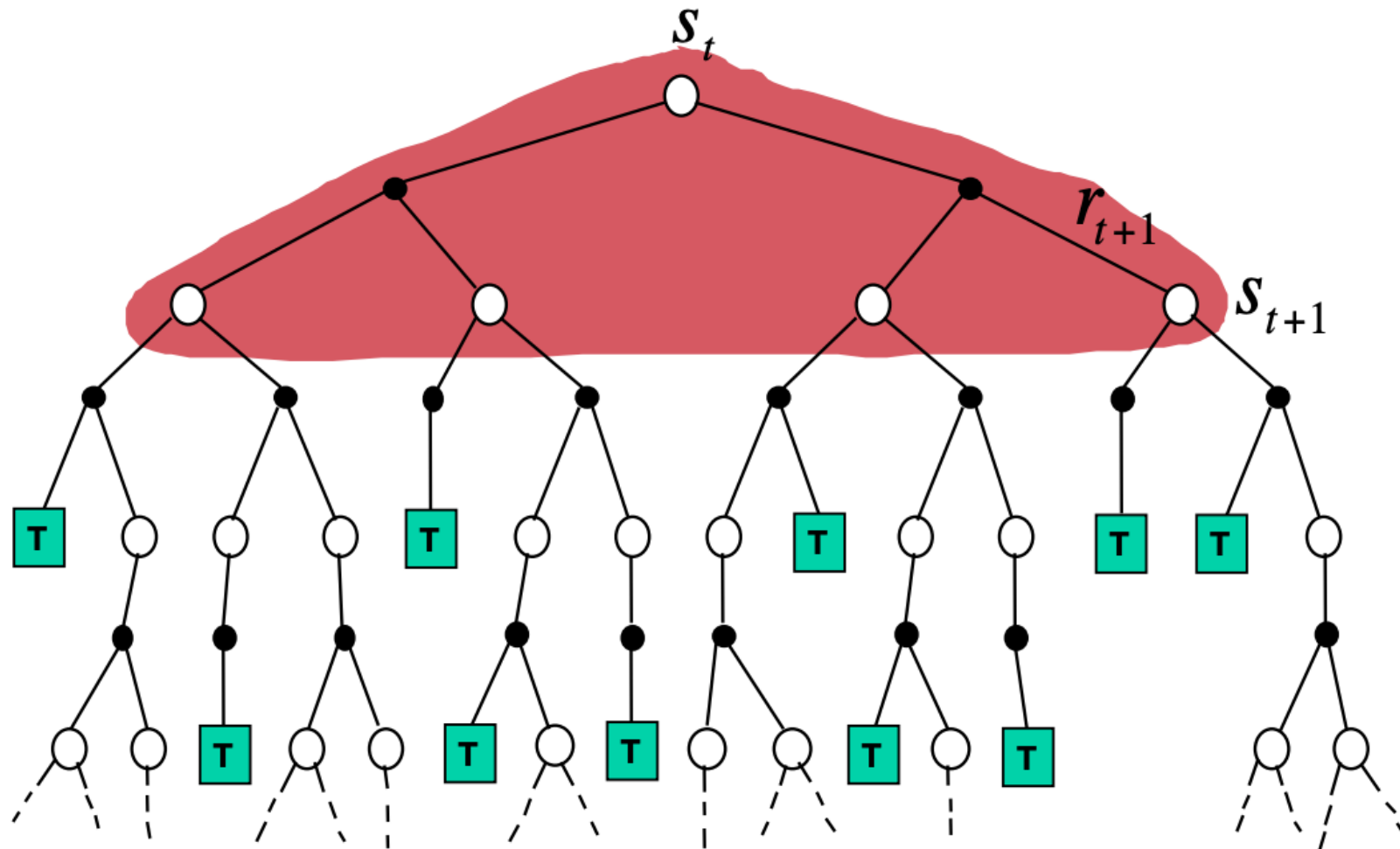
Temporal Difference Backup

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$



Dynamic Programming Backup

$$V(S_t) \leftarrow \mathbb{E}_{\pi} [R_{t+1} + \gamma V(S_{t+1})]$$



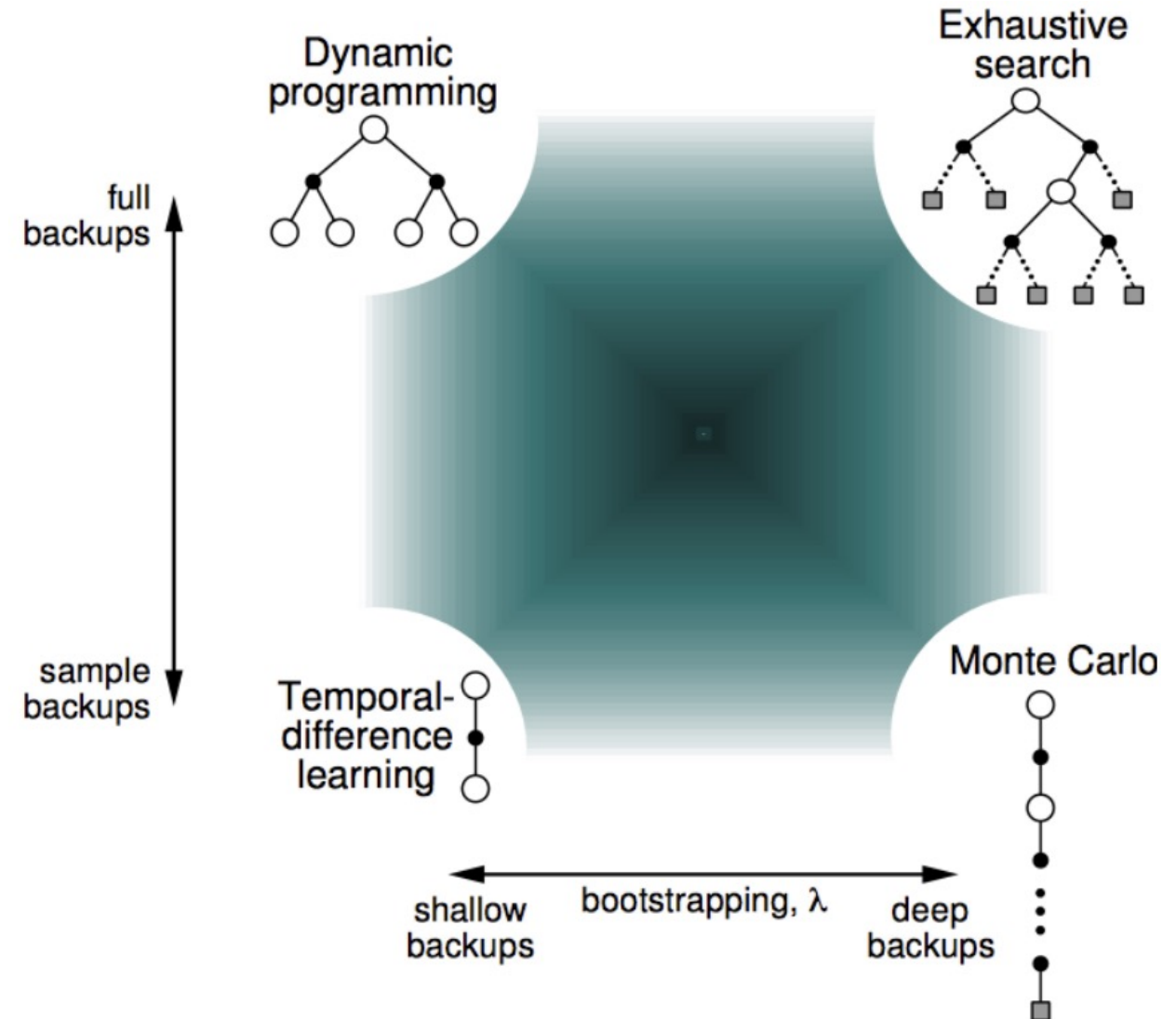
Unified View of Reinforcement Learning

Bootstrap (update involves an estimate)

- MC does not bootstrap
- DP bootstraps
- TD bootstraps

Sampling (use samples to estimate expectation)

- MC samples
- DP does not sample
- TD samples

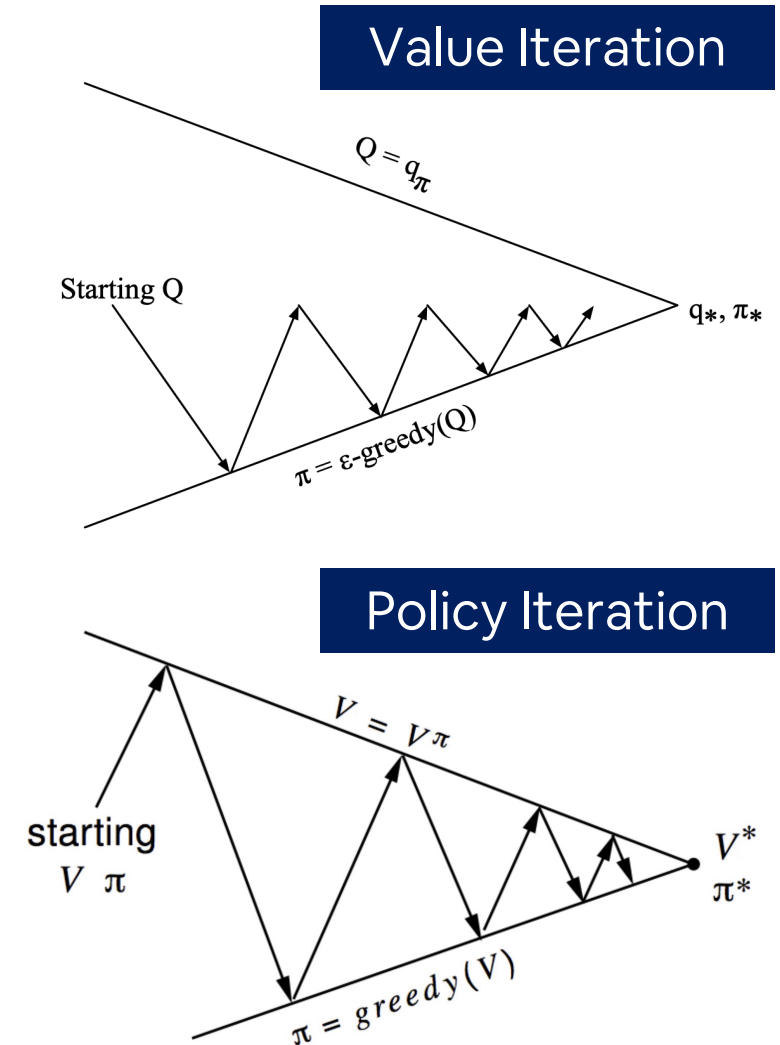


TD-Learning Control

- (On Policy) SARSA
- (Off Policy) Q-Learning

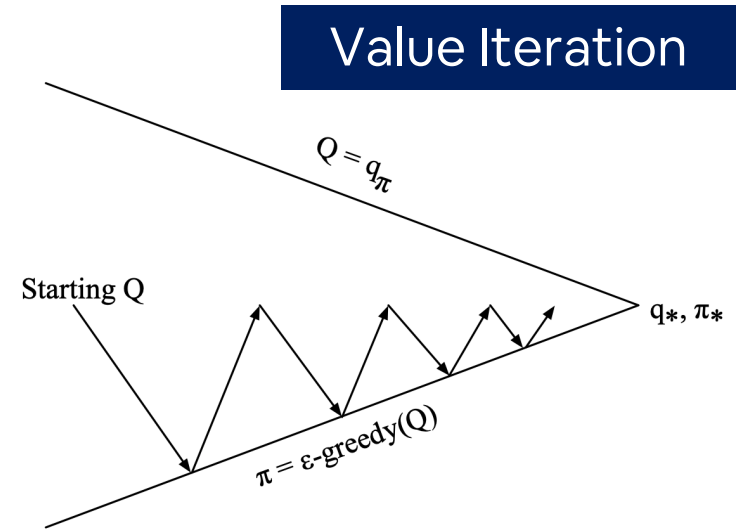
Control: Model-free Generalized Policy Iteration (GPI) with TD learning

- We apply again the GPI approach (iterations between **prediction** and improvement) for solving **control**
- Since we are model-free, we use interactions over q_π
- With TD(0) we need a way to handle incomplete sequences and we will consider improvements over the TD target (updates at every step of the episode)
- We start by considering the on-policy case



Control: Model-free Generalized Policy Iteration (GPI) with TD learning

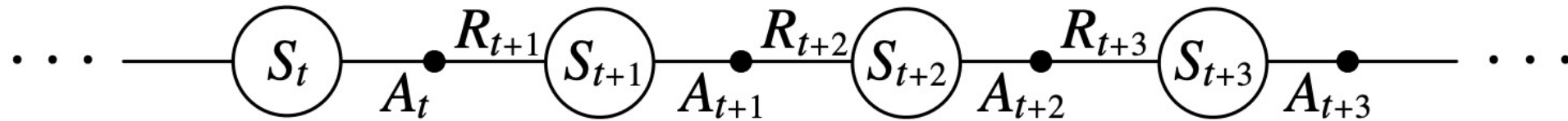
- We apply again the GPI approach (iterations between **prediction** and improvement) for solving **control**
- Since we are model-free, we use interactions over q_π
- With TD(0) we need a way to handle incomplete sequences and we will consider improvements over the TD target (updates at every step of the episode)
- We start by considering the on-policy case



We'll consider:

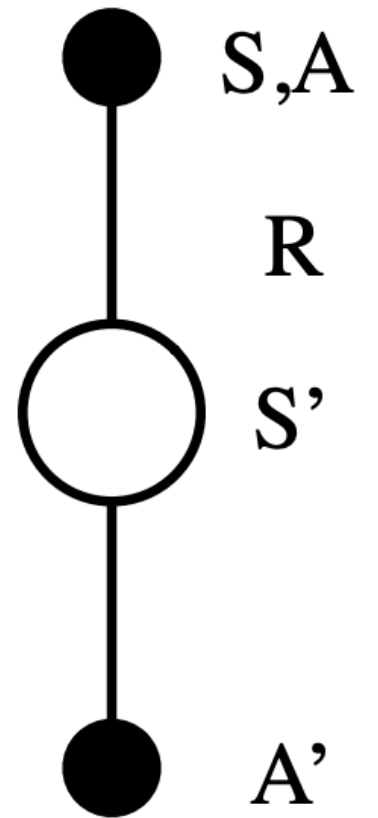
- Similarly to value iterations, only partial evaluation of q_π (in line with the principle of TD(0) of using the 'newest' estimation)
- As in MC, we can consider approaches for dealing with exploration, like ϵ -greedy

Control: SARSA - On-policy TD learning for Control

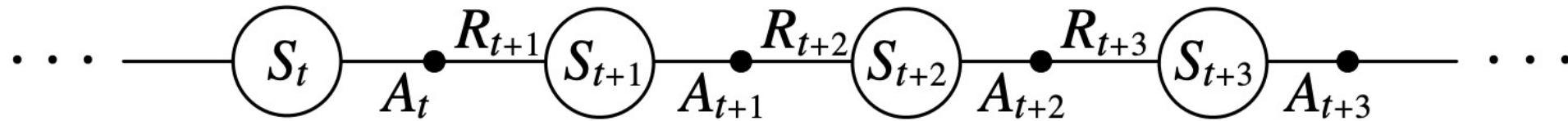


- We need to consider transactions from (state, action) to (state, action)
- TD target
- TD error

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

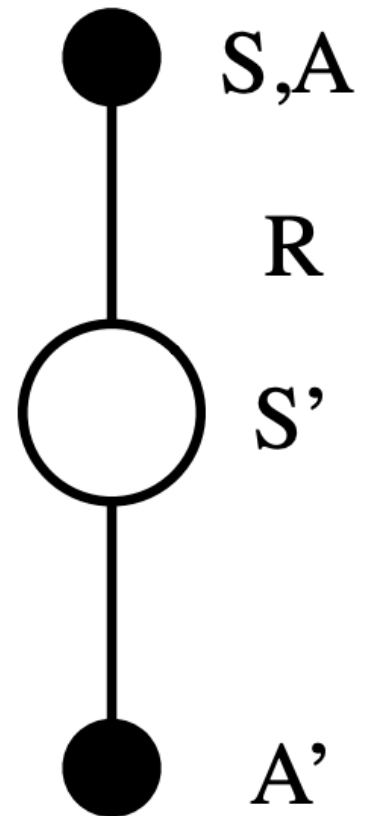


Control: SARSA - On-policy TD learning for Control

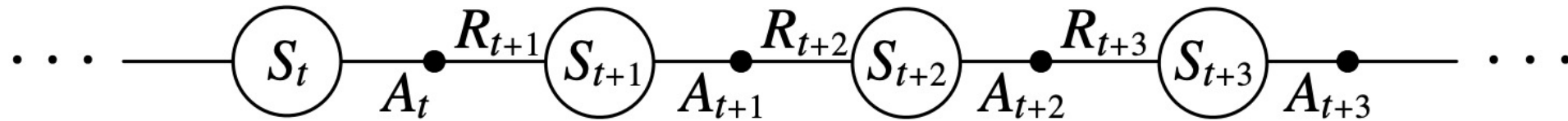


- We need to consider transactions from (state, action) to (state, action)
- TD target
- TD error

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$



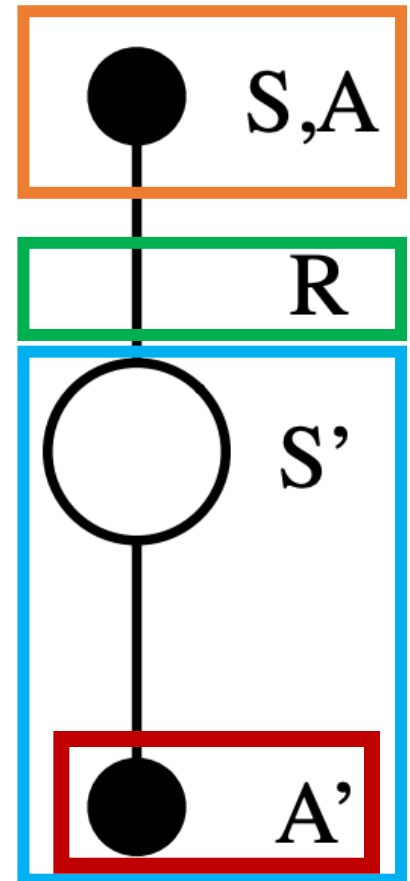
Control: SARSA - On-policy TD learning for Control



- We need to consider transactions from (state, action) to (state, action)
- TD target
- TD error

Pay attention: A' (A_{t+1}) is taken accordingly to your policy π

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$



Control: SARSA - On-policy TD learning for Control

Which elements will be on the algorithm?

Control: SARSA - On-policy TD learning for Control

Which elements will be on the algorithm?

- GPI: policy evaluation + policy improvement
- Since we are doing TD learning, for loops both over the various episodes (we are model-free, we need data) and over the various steps in an episode
- In TD learning we will also need to consider incremental updates (so we need to set up an α parameter)
- For exploration we may consider epsilon greedy approach

Control: SARSA - On-policy TD learning for Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Loop for each step of episode:

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

Control: SARSA - On-policy TD learning for Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialization: α is an hyperparameter

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Loop for each step of episode:

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

until S is terminal

Control: SARSA - On-policy TD learning for Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

We are considering ε -greedy to ensure exploration

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Loop for each step of episode:

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

until S is terminal

Control: SARSA - On-policy TD learning for Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Loop for each step of episode:

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ε -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

until S is terminal

In TD we always consider a double loop where we make updates for each step in each episode!

Control: SARSA - On-policy TD learning for Control

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except

Loop for each episode:

Initialize S

Choose A from S using policy derived from Q (e.g., ε -greedy)

Loop for each step of episode:

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ε -greedy)

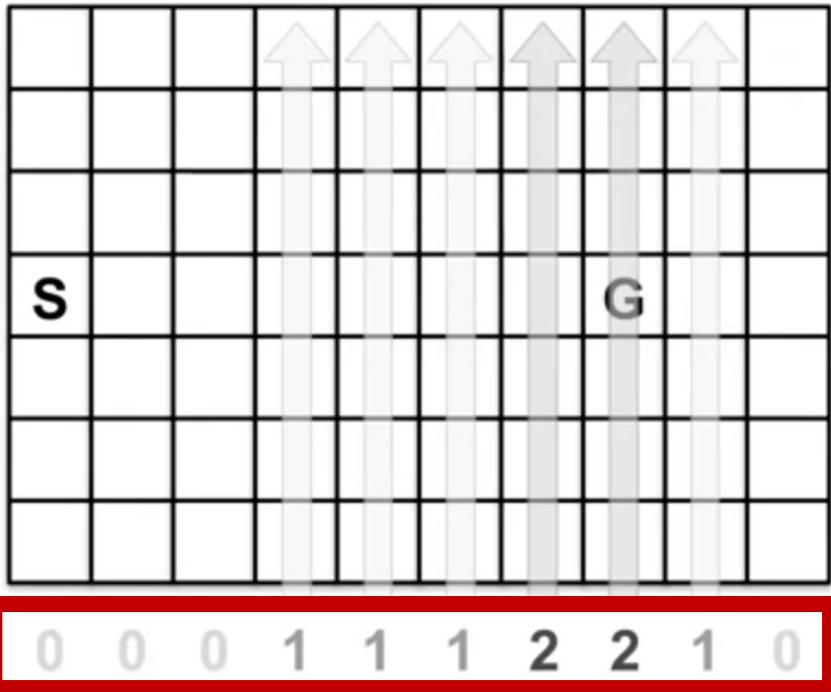
$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$ These are just to 'move' for next steps $(S', A') \rightarrow (S, A)$

until S is terminal

We actually perform the next action, according to the policy, and then update Q . We will act epsilon greedily on Q at next step!

Control: SARSA – Windy Grid World Example

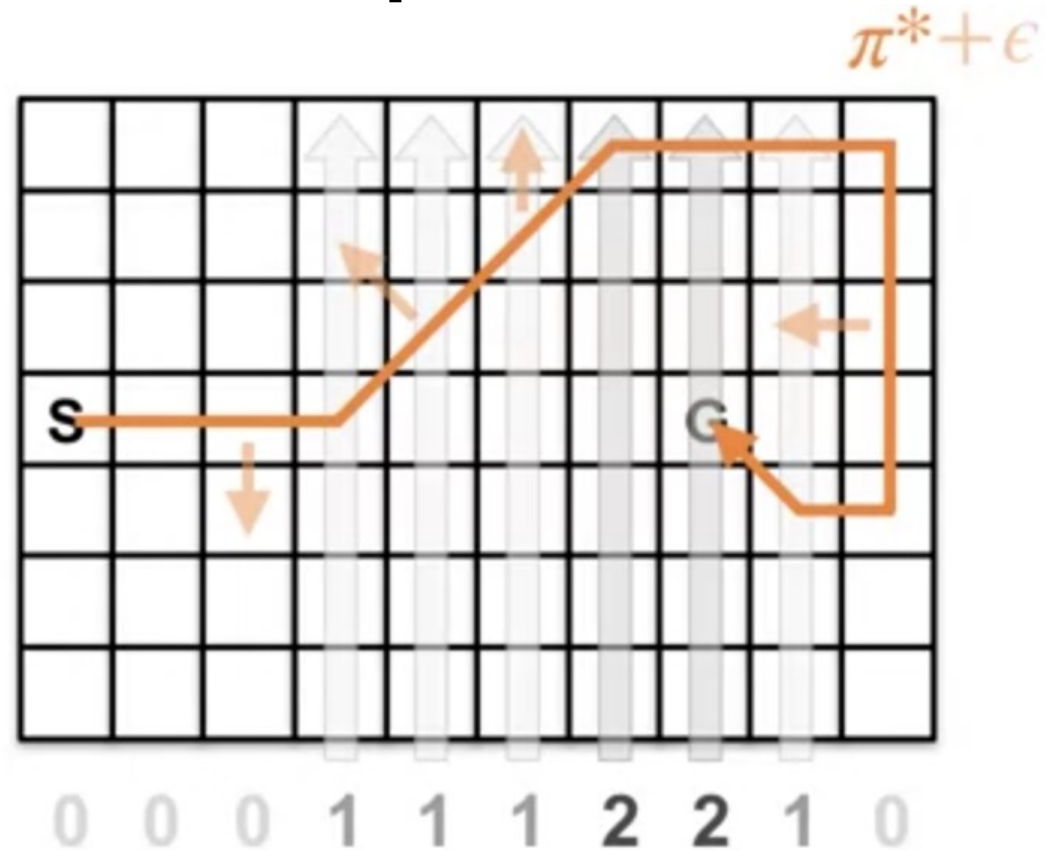


Actions

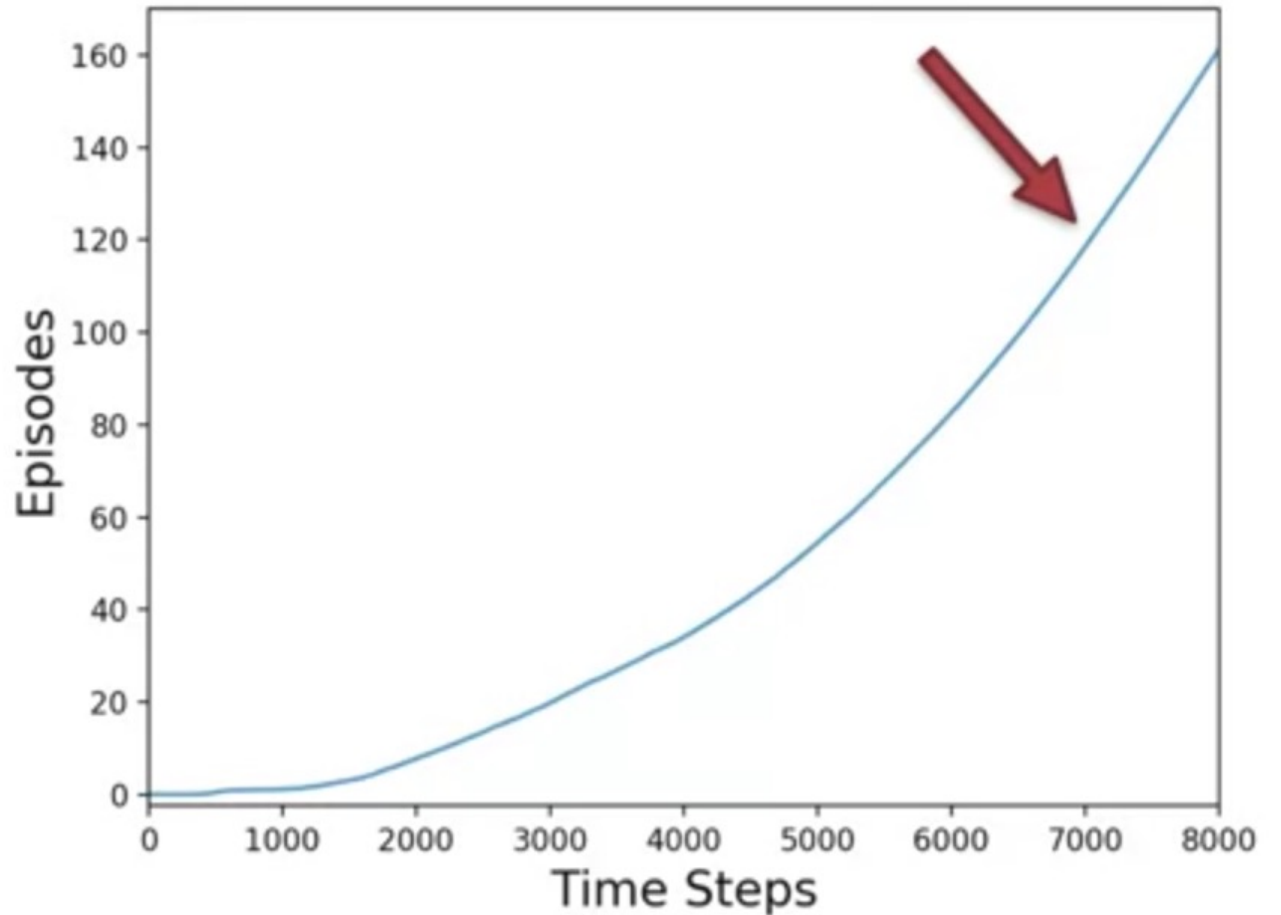
$$R_{step} = -1$$
$$\gamma = 1$$

- The wind brings the agent up in the column of a number of cells equal to the **number reported in the bottom**
- Going outside the grid world does nothing
- This is a case where MC doesn't work well since many episodes may not end

Control: SARSA – Windy Grid World Example



Sarsa: $\epsilon = 0.1$
 $\alpha = 0.5$



Control: Q-learning – (Off-policy) TD learning for Control

- Q-learning is the most popular approach to RL control
- We'll see how Q-learning (for TD(0)):
 1. Can be derived as a slight modification from SARSA
 2. Is associated with the Bellman Optimality Equation
 3. (Can be considered off-policy)

Control: Q-learning vs. SARSA

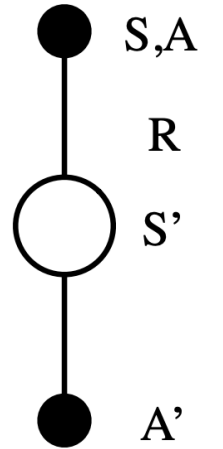
SARSA

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

$S \leftarrow S'; A \leftarrow A';$



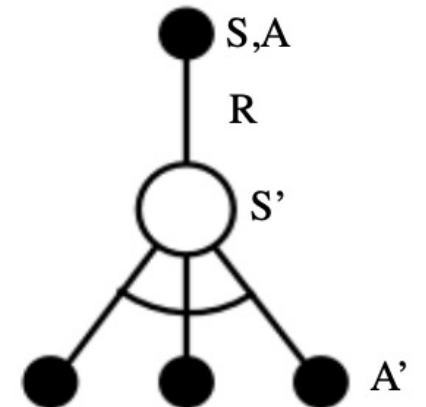
Q-learning

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$S \leftarrow S'$



Control: Q-learning vs. SARSA

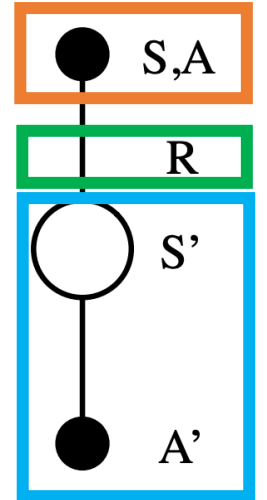
SARSA

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

$$S \leftarrow S'; A \leftarrow A';$$



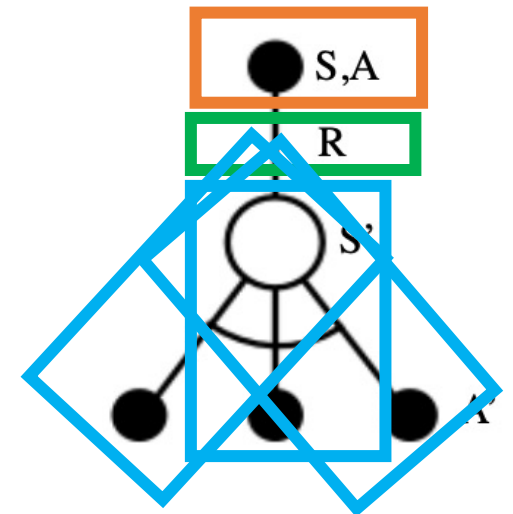
Q-learning

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$$S \leftarrow S'$$



Control: Q-learning vs. SARSA

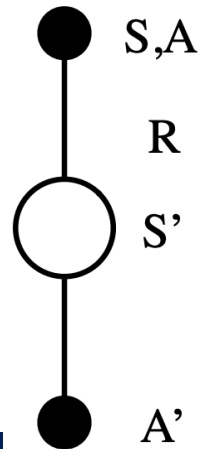
SARSA

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

$S \leftarrow S'; A \leftarrow A';$



Q-learning

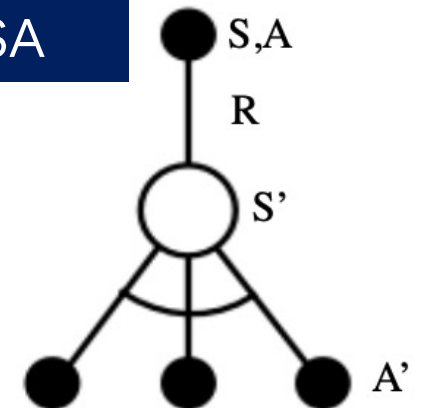
Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$S \leftarrow S'$

Q-learning is usually faster than SARSA



Control: Q-learning vs. SARSA

SARSA

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

$S \leftarrow S'; A \leftarrow A';$

You actually perform next action, according to the policy and then update $Q(s,a)$

Q-learning

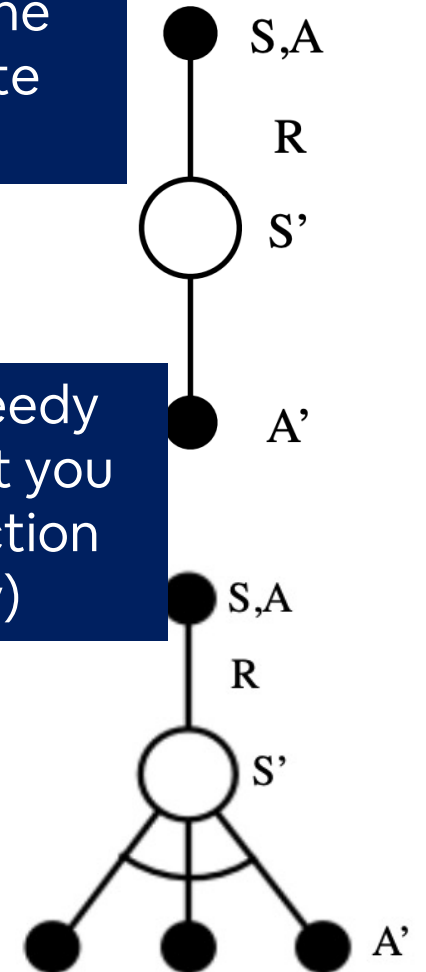
Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$S \leftarrow S'$

You look ahead and imagine greedy next action to update $Q(s,a)$ (but you then perform the actual next action based on your current policy)



Control: Q-learning – (Off-policy) TD learning for Control

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

 until S is terminal

Control: Q-learning – (Off-policy) TD learning for Control

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$

Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, arbitrarily except that $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

 Initialize S

 Loop for each step of episode:

 Choose A from S using policy derived from Q (e.g., ε -greedy)

 Take action A , observe R, S'

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

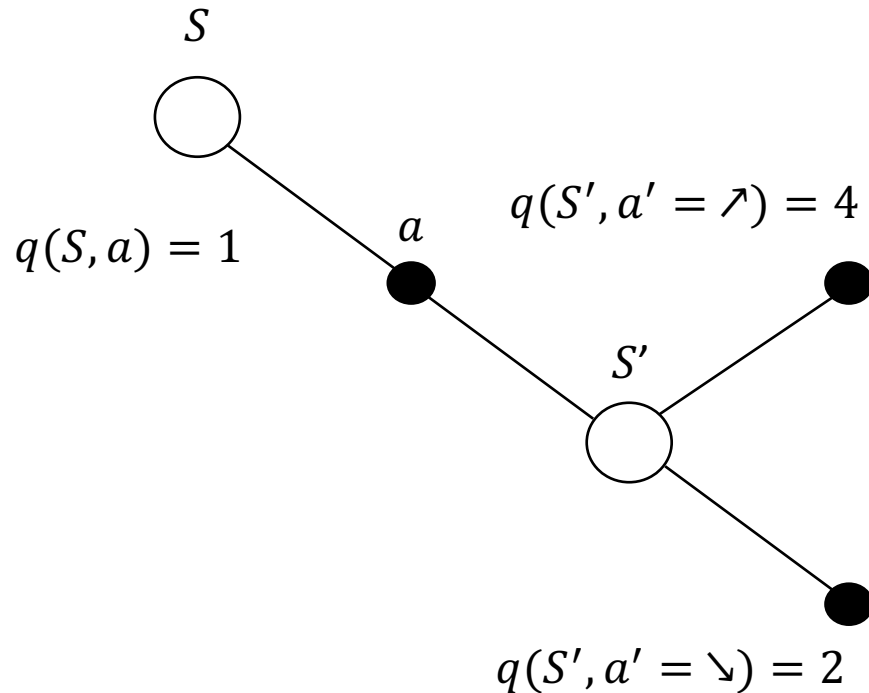
$S \leftarrow S'$

 until S is terminal

Control: Q-learning vs. SARSA – example

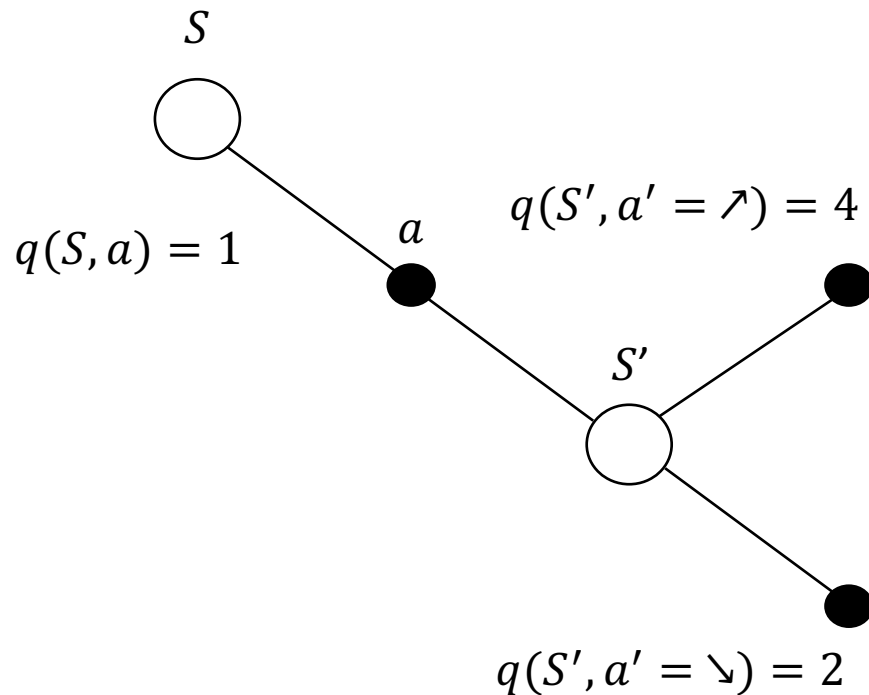
$$\gamma = 0.5$$
$$\alpha = 0.1$$

First episode we transition from S to S' by taking action a and we get a reward of +1



Control: Q-learning vs. SARSA – example

$$\gamma = 0.5$$
$$\alpha = 0.1$$



First episode we transition from S to S' by taking action a and we get a reward of +1

SARSA:

- Target:

$r + \gamma q(s', \nearrow) = +1 + 0.5(+4) = +3$ if by policy π we have $a' = \nearrow$ in s'

$r + \gamma q(s', \searrow) = +1 + 0.5(+2) = +2$ if by policy π we have $a' = \searrow$ in s'

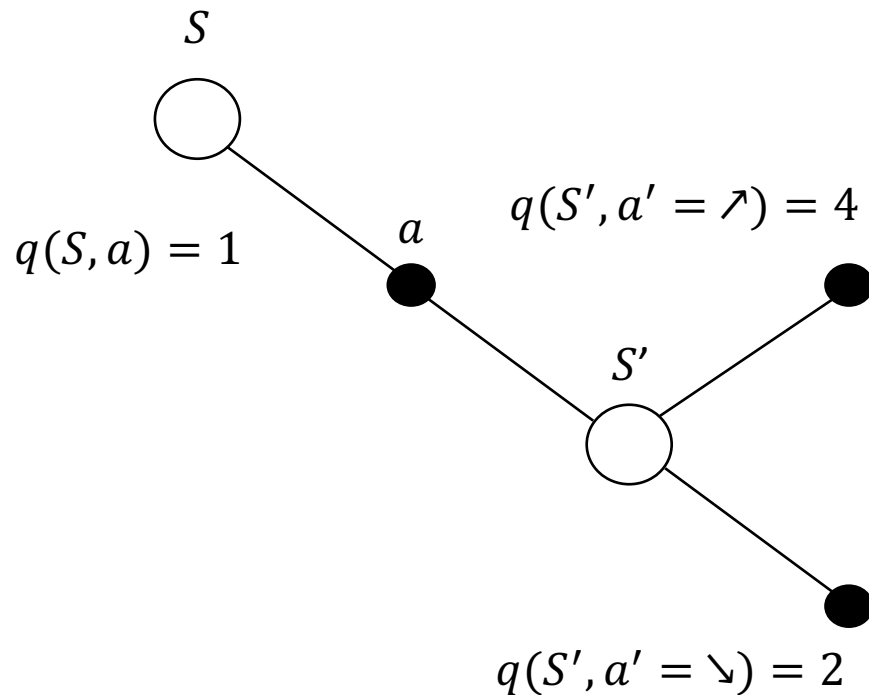
- Update

$q(S, a) = 1 + 0.1 * (3 - 1) = 1.2$ if by policy π we have $a' = \nearrow$ in s'

$q(S, a) = 1 + 0.1 * (2 - 1) = 1.1$ if by policy π we have $a' = \searrow$ in s'

Control: Q-learning vs. SARSA – example

$$\gamma = 0.5$$
$$\alpha = 0.1$$



First episode we transition from S to S' by taking action a and we get a reward of +1

SARSA:

- Target:

$$r + \gamma q(s', \nearrow) = +1 + 0.5(+4) = +3 \text{ if by policy } \pi \text{ we have } a' = \nearrow \text{ in } s'$$

$$r + \gamma q(s', \searrow) = +1 + 0.5(+2) = +2 \text{ if by policy } \pi \text{ we have } a' = \searrow \text{ in } s'$$

- Update

$$q(S, a) = 1 + 0.1 * (3 - 1) = 1.2 \text{ if by policy } \pi \text{ we have } a' = \nearrow \text{ in } s'$$

$$q(S, a) = 1 + 0.1 * (2 - 1) = 1.1 \text{ if by policy } \pi \text{ we have } a' = \searrow \text{ in } s'$$

Q-learning

- Target:

$$r + \gamma \max_{a'} q(s', a') = +1 + 0.5(+4) = +3 \text{ independently from the current policy } \pi \text{ (for this reason it is off-policy!)}$$

- Update:

$$q(S, a) = 1 + 0.1 * (3 - 1) = 1.2$$

Control: Q-learning vs. SARSA and connection with Dynamic Programming

Sarsa: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$

Bellman
Expectation
Equation

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) \left(r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right)$$

Q-learning: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$

Bellman
Optimality
Equation

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) \left(r + \gamma \max_{a'} q_{\pi}(s', a') \right)$$

Control: Q-learning vs. SARSA and connection with Dynamic Programming

Sarsa: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$

Bellman
Expectation
Equation

$$q_{\pi}(s, a) = \sum_{s', r} p(s', r | s, a) \left(r + \gamma \sum_{a'} \pi(a' | s') q_{\pi}(s', a') \right)$$

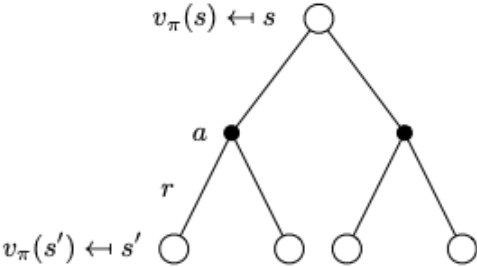
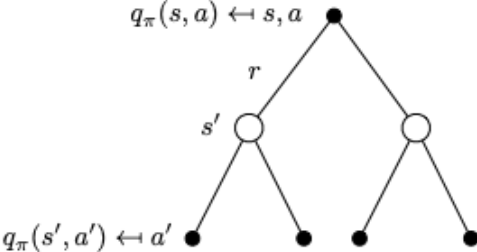
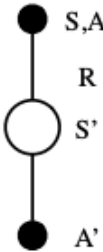
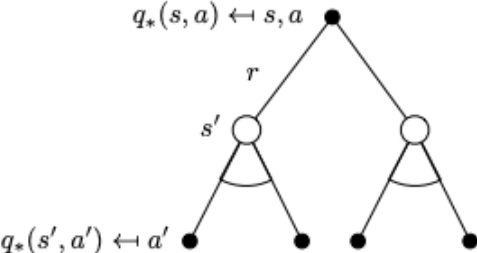
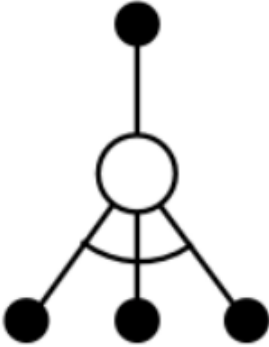

SARSA is a
sample-based
version of Policy
Iteration

Q-learning: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$

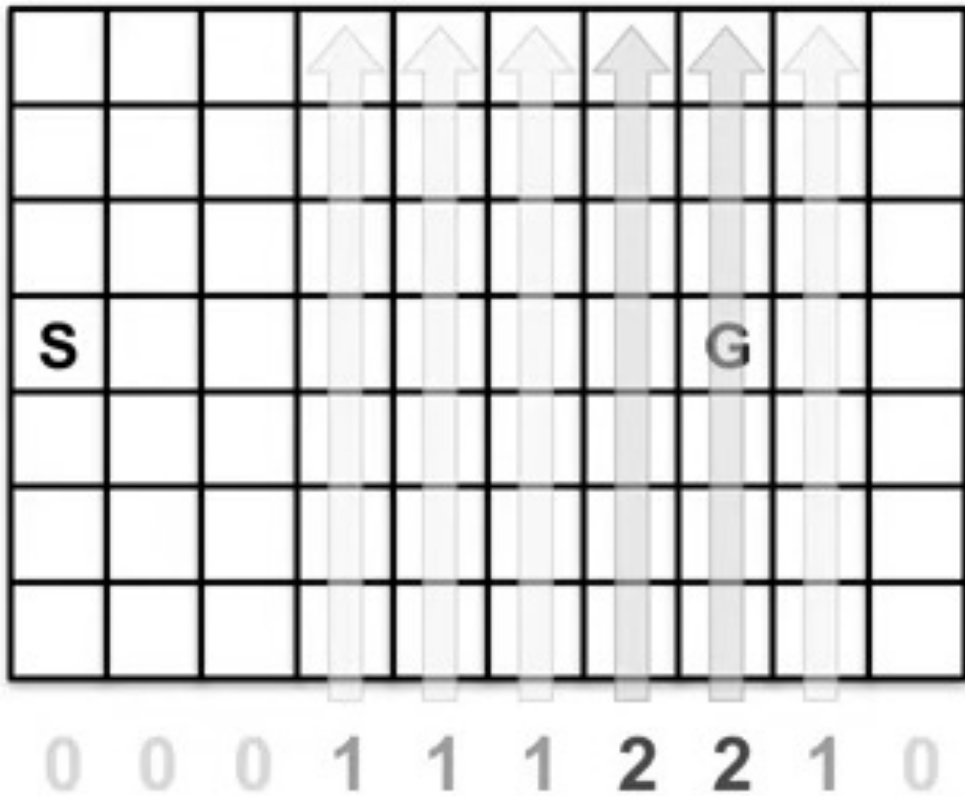
Bellman
Optimality
Equation

$$q_*(s, a) = \sum_{s', r} p(s', r | s, a) \left(r + \gamma \max_{a'} q_{\pi}(s', a') \right)$$

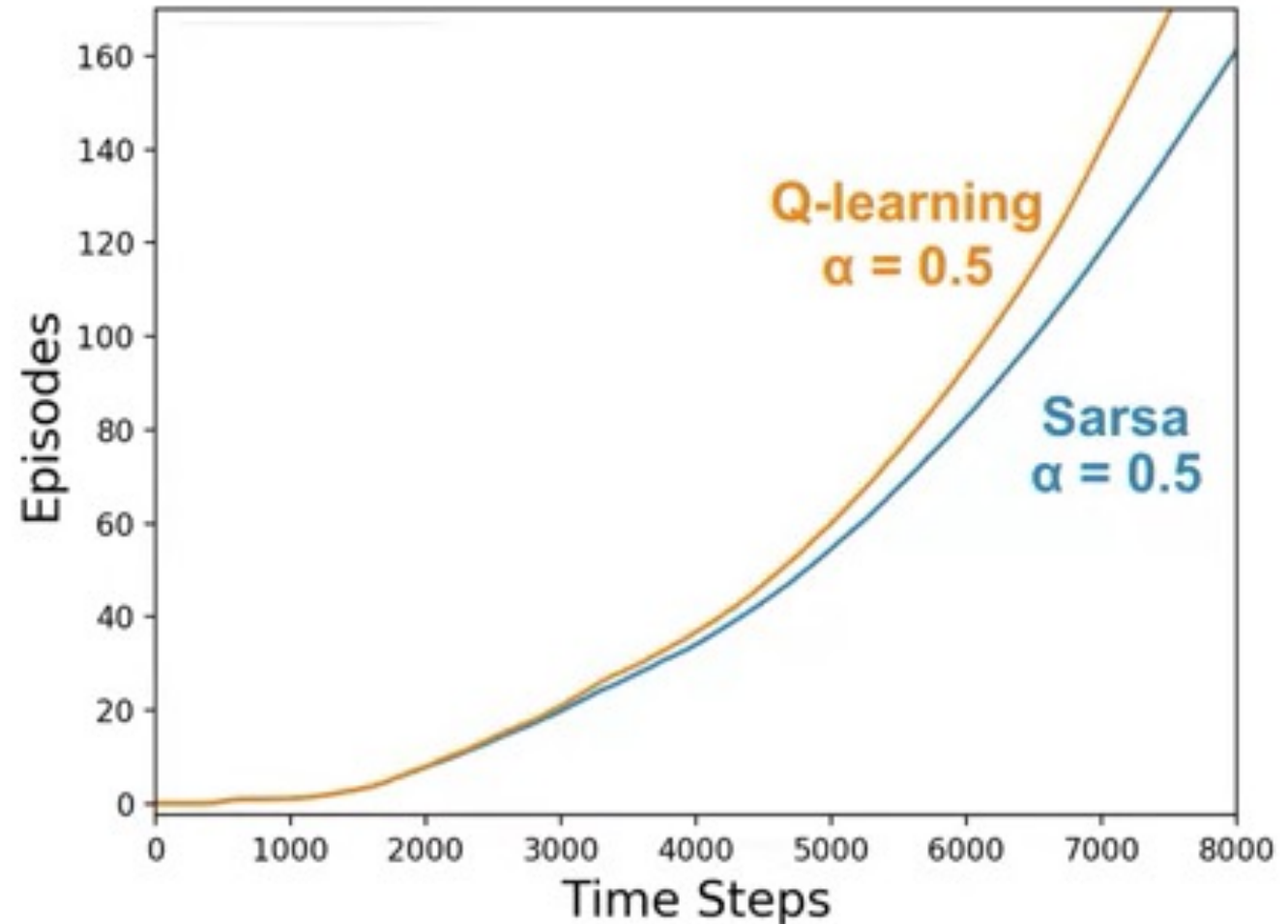

Q-learning is a
sample-based
version of Value
Iteration

	<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>
Bellman Expectation Equation for $v_{\pi}(s)$	 Iterative Policy Evaluation	 TD Learning
Bellman Expectation Equation for $q_{\pi}(s, a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_{*}(s, a)$	 Q-Value Iteration	 Q-Learning

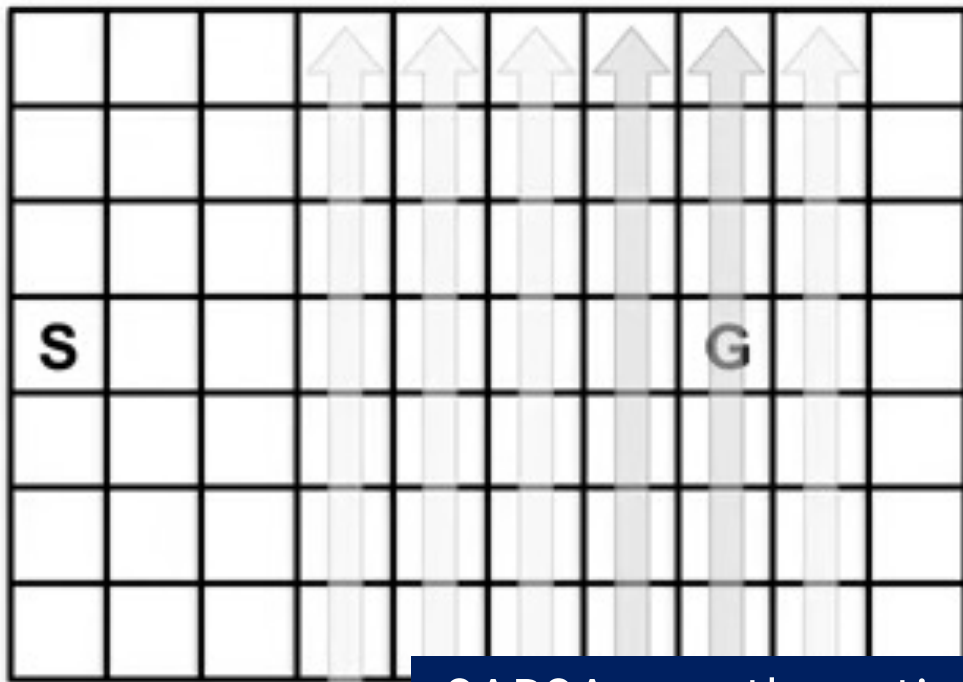
Control: SARSA vs Q-learning – Windy Grid World Example



$\epsilon = 0.1$



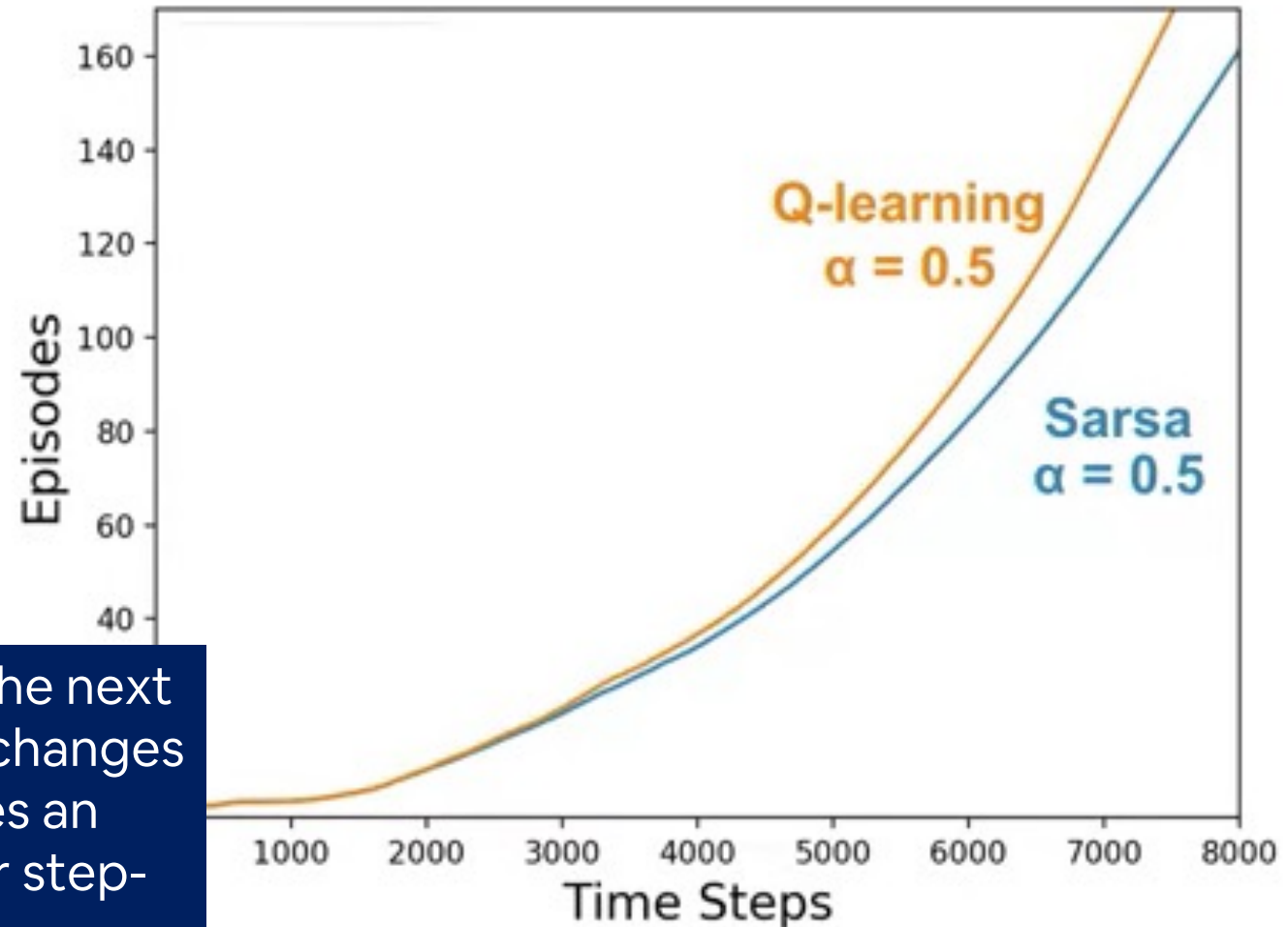
Control: SARSA vs Q-learning – Windy Grid World Example



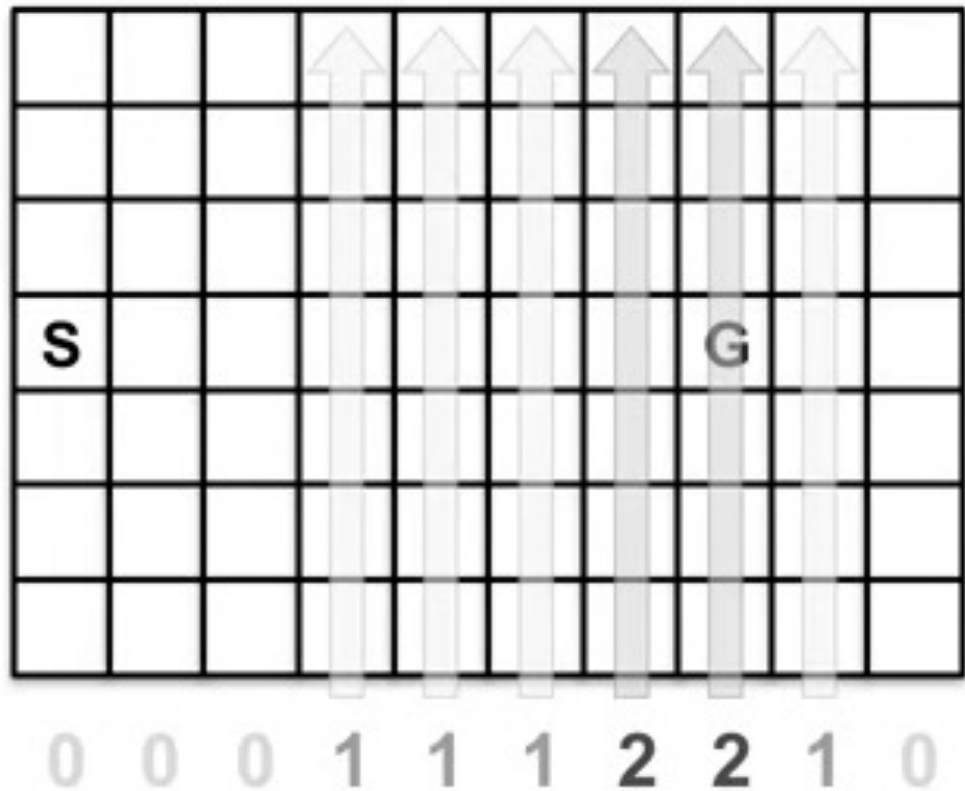
0 0 0 1

ϵ

SARSA uses the estimate of the next action value in its target: this changes every time the agent takes an exploratory action. A smaller step-size can help SARSA

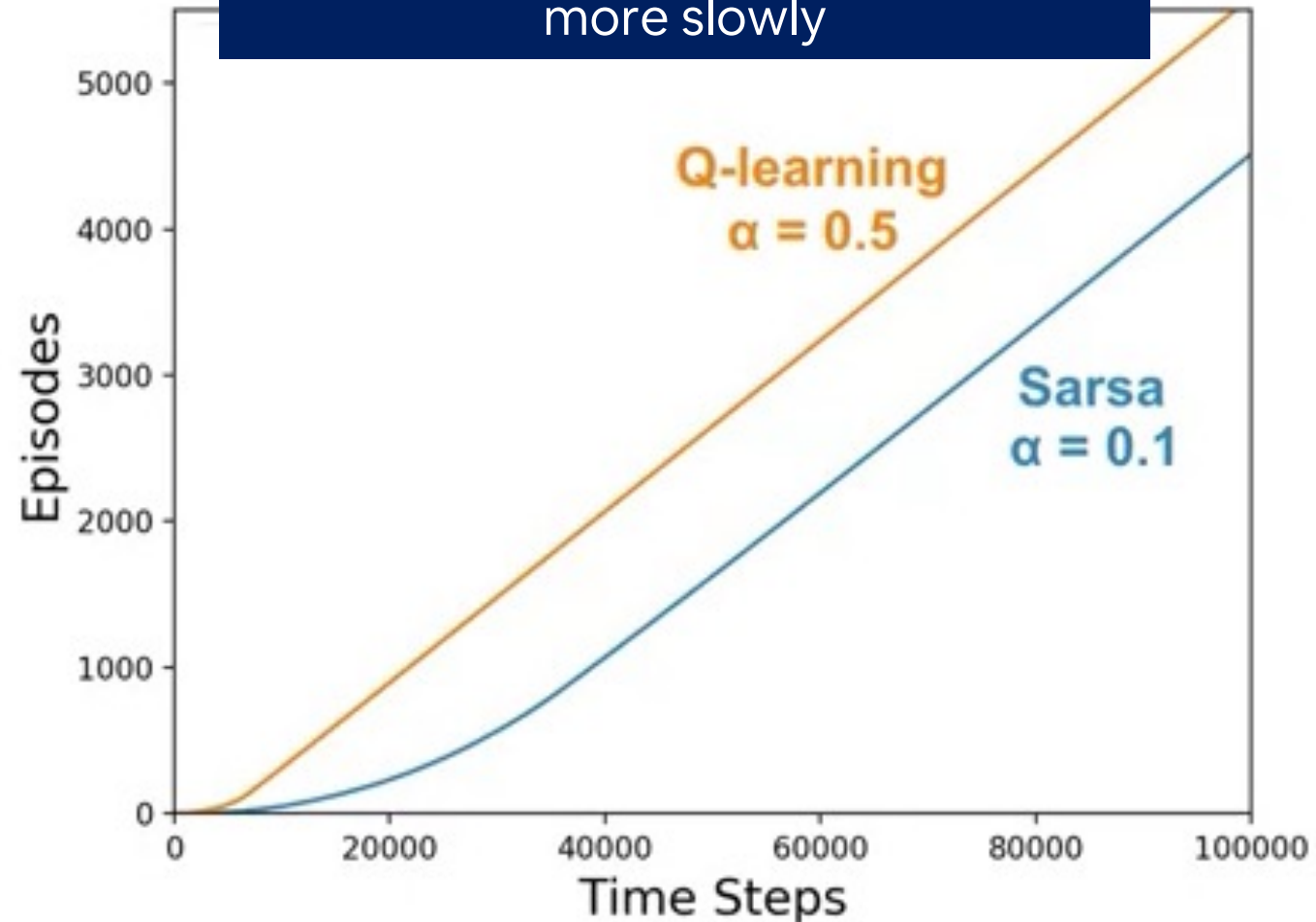


Control: SARSA vs Q-learning – Windy Grid World Example



$\epsilon = 0.1$

SARSA finds the same solution of Q-learning (see the final slope), but more slowly



Control: why is Q-learning off-policy?

SARSA

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

$S \leftarrow S'; A \leftarrow A';$

You actually perform next action, according to the policy and then update $Q(s,a)$

Q-learning

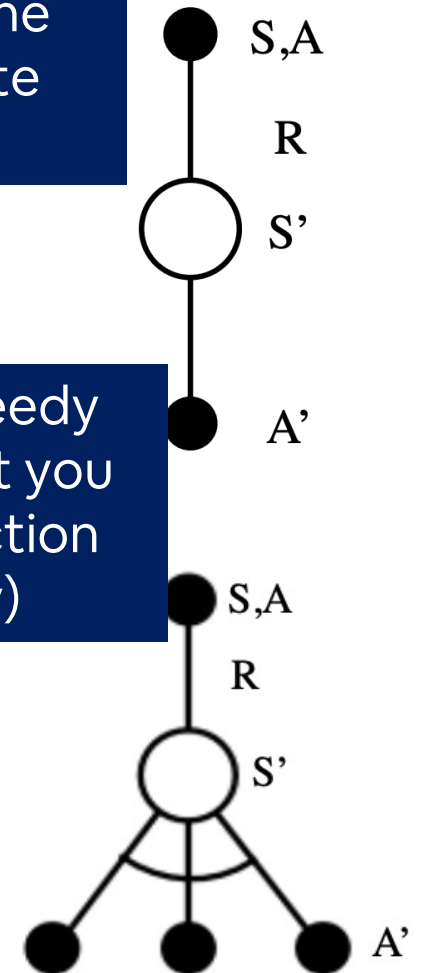
Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$S \leftarrow S'$

You look ahead and imagine greedy next action to update $Q(s,a)$ (but you then perform the actual next action based on your current policy)



Control: why is Q-learning off-policy?

SARSA

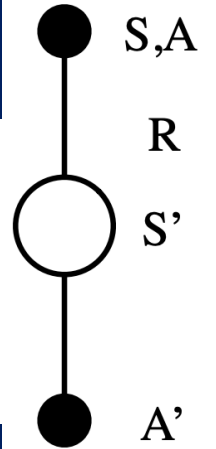
We only have one (target) policy here

Take action A , observe R, S'

Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

$S \leftarrow S'; A \leftarrow A';$



Q-learning

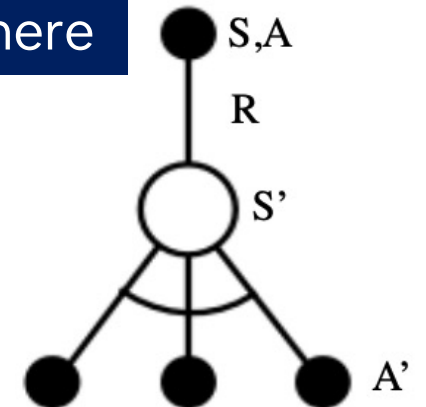
We have a behaviour (epsilon-greedy) and a target policy (greedy!) here

Choose A from S using policy derived from Q (e.g., ϵ -greedy)

Take action A , observe R, S'

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$$

$S \leftarrow S'$



Control: why is Q-learning off-policy?

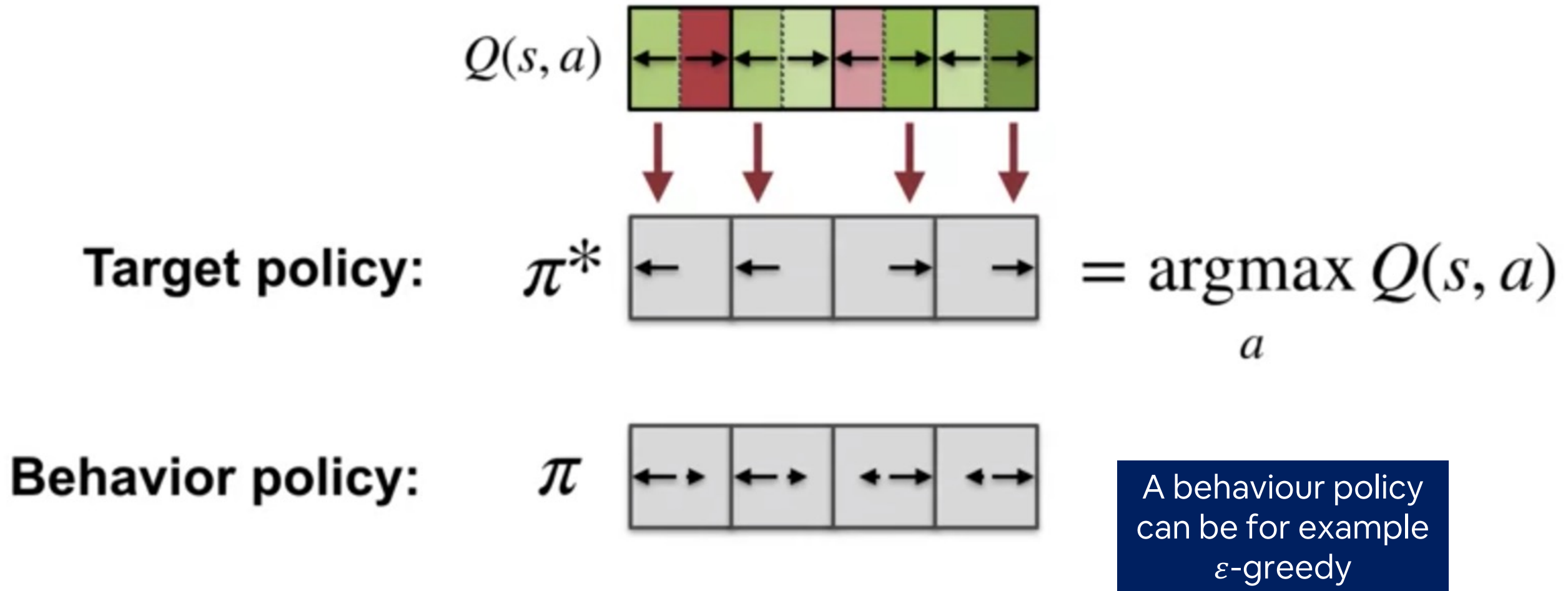
Sarsa: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t))$

$\sim \pi$

Q-learning: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t))$

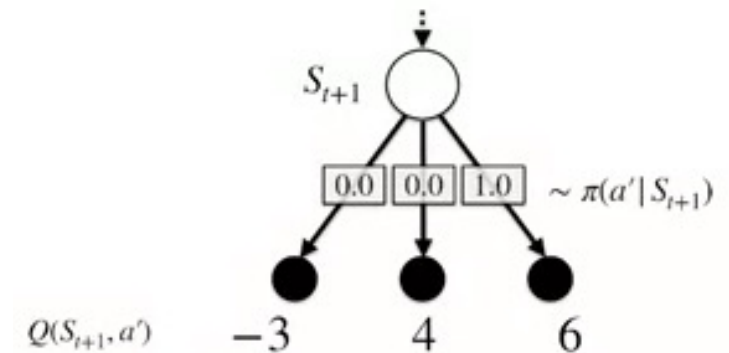
$\sim \pi_* \neq \pi$

Control: why is Q-learning off-policy?



Control: why is Q-learning off-policy?

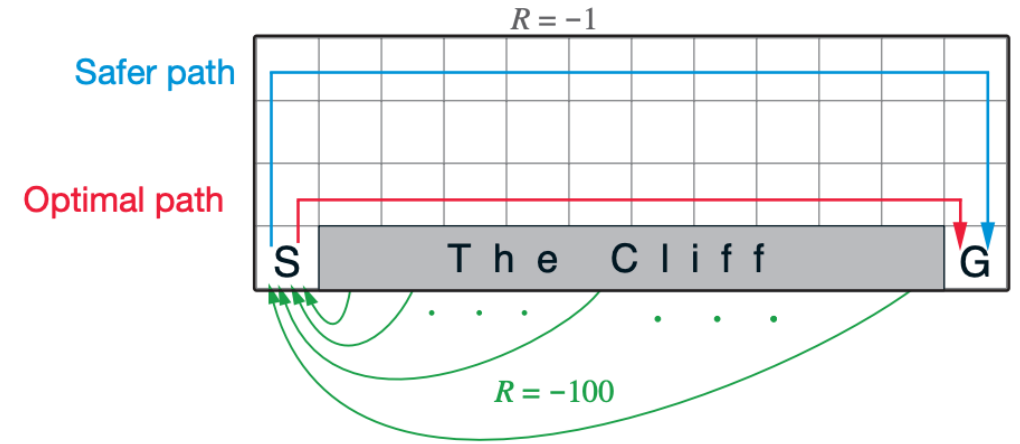
- **No importance sampling is required:** it is because the agent is estimating action values with unknown policy and it does not need importance sampling ratios to correct for the difference in action selection.
- The action value function represents the returns following each action in a given state: the agent's target policy represents the probability of taking each action in a given state.
- Putting these two elements together, the agent can calculate the expected return under its target policy from any given state,
- Q-learning uses exactly this technique to learn off-policy.
- Since the agent's target policies are greedy, with respect to its action values, all non-maximum actions have probability 0.
- As a result, the expected return from that state is equal to a maximal action value from that state.



$$\sum_{a'} \pi(a' | S_{t+1}) Q(S_{t+1}, a') = \mathbb{E}_\pi[G_{t+1} | S_{t+1}] = \max_{a'} Q(S_{t+1}, a') = 6$$

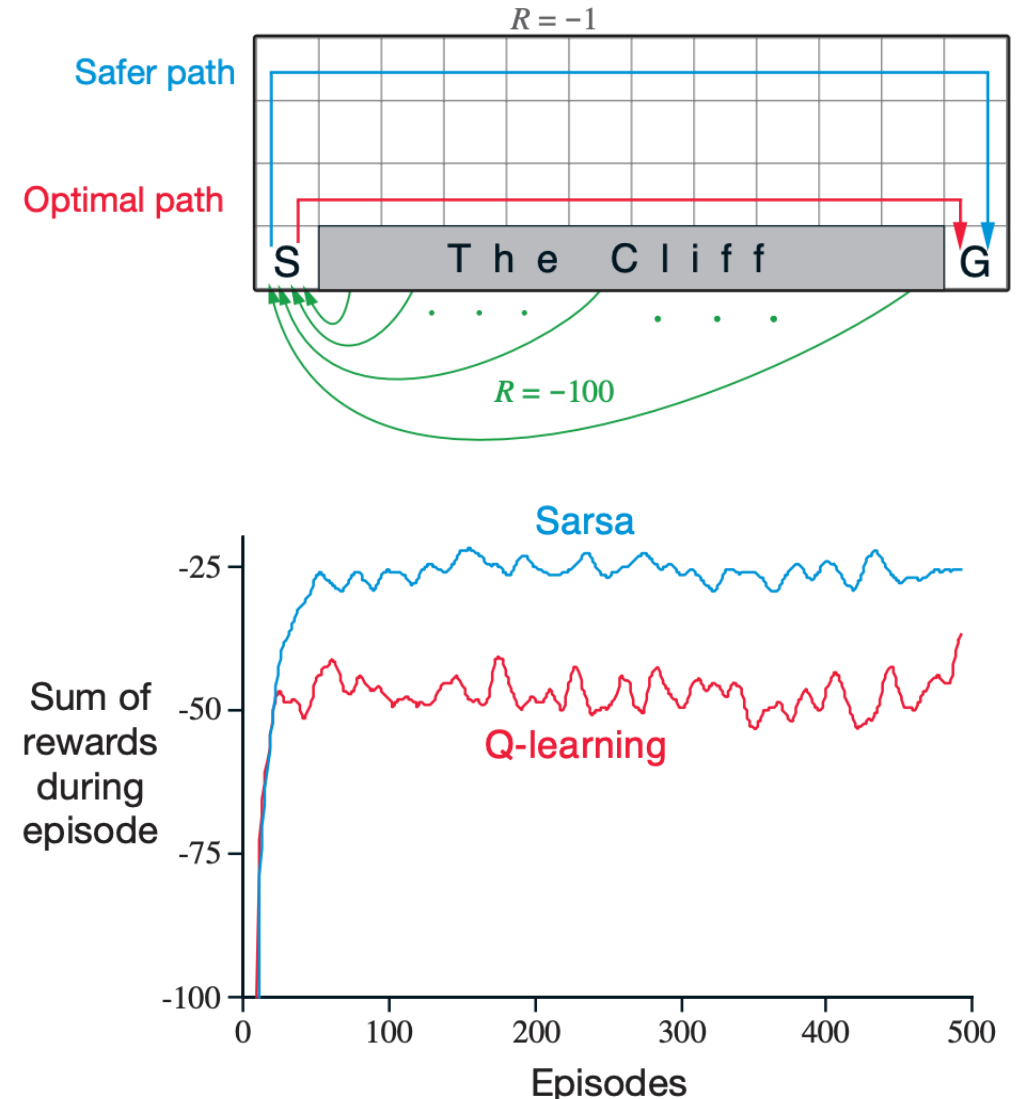
Control: why is Q-learning off-policy?

- Q-learning doesn't iterate between policy evaluation and policy improvement, but rather learns the optimal values directly. Not always ideal!



Control: why is Q-learning off-policy?

- Q-learning doesn't iterate between policy evaluation and policy improvement, but rather learns the optimal values directly. Not always ideal!
- Since Q-learning learns the optimal value function, it quickly learns that an optimal policy travels right alongside the cliff.
- However, since his actions are epsilon greedy, traveling alongside the cliff occasionally results and falling off of the cliff.
- Sarsa learns about his current policy, taking into account the effect of epsilon greedy action selection.



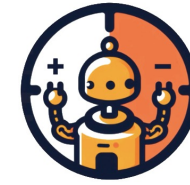
Credits

- Image of the course is taken from C. Mahoney 'Reinforcement Learning' <https://towardsdatascience.com/reinforcement-learning-fda8ff535bb6>
- Unified view of RL was taken from D. Silver 'Reinforcement Learning' course @ UCL



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Reinforcement Learning 2024/2025



RL2
Reinforcement Learning
Research Lab @ UniPD

Thank you! Questions?

Gian Antonio Susto
Ruggero Carli

